

UNIVERSIDAD ESTATAL PENÍNSULA DE SANTA ELENA
FACULTAD SISTEMAS Y TELECOMUNICACIONES
CARRERA DE SOFTWARE

ASIGNATURA:
SISTEMAS DISTRIBUIDOS Y CLOUD COMPUTING

ACTIVIDAD

TEMA:
RETO TÉCNICO: PEDIDOS, PAGOS SIMULADOS E IDEMPOTENCIA

CURSO:
SEPTIMO SEMESTRE 7/1

ELABORADO POR:
ANDY BRYAN ALEJANDRO VERA
ALISSON YAMEL REYES RICARDO
YANDRIS MIGUEL RIVERA TORRES

DOCENTE:
ING. ANTHONY ABRAHAN PACHAY ESPINOZA

FECHA:
31 DE OCTUBRE DEL 2026.

PERIODO 2026 – 2

Reto técnico: pedidos, pagos simulados e idempotencia

Order API y Payment Mock API con Flask + SQLite · Pruebas con Postman

Repositorio	https://github.com/Ali-Rr26/idempotent-orders-payments
Video (3–5 min)	Pegar el enlace del video (verificar que abra en una ventana de incógnito)
Colección de Postman	`postman/idempotent-orders-payments.postman_collection.json` (en el repositorio)

1. Solución

Dos servicios Flask independientes, cada uno con su propia base SQLite. Ambos se ejecutan en local (dos terminales) o con `docker compose up --build`; las instrucciones completas están en el README del repositorio.

```
Postman ──► Order API :5000 (orders.db) ──► Payment Mock API :5001 (payments.db)
```

- **Order API**: recibe el pedido con un header `Idempotency-Key`, reserva el stock y crea el pedido en una sola transacción. Después cobra al mock y expone el estado del pedido. Tablas: `products`, `orders` (clave `UNIQUE`), `reservations` (una por pedido, `UNIQUE`).
- **Payment Mock API**: registra un cobro por clave (`payment_key UNIQUE`) y simula tres modos. `approve` y `reject` responden de inmediato. `timeout` hace **commit** del cobro y luego tarda 5 s en responder; la Order API corta a los 2 s. Expone `GET /payments/{key}` para consultar el resultado.
- La clave de pago es estable: `payment_key = "pay-" + Idempotency-Key`. Si la Order API reenvía el cobro, el mock devuelve el registro existente y no crea otro.
- Los errores usan el formato estándar RFC 9457 (`application/problem+json`) con un mensaje explicativo; por ejemplo, el 422 cuando se reutiliza una clave con otro cuerpo.

Estados del pedido

Estado	Significado	Reserva
`PENDING_PAYMENT`	Pedido creado y stock reservado; el cobro está en curso.	`RESERVED`
`PAYMENT_UNCERTAIN`	La respuesta del pago se perdió (timeout, error de red o 5xx). No se sabe si hubo cobro: hay que consultar al mock, no volver a cobrar.	`RESERVED`
`PAID`	Final. Pago aprobado (directo o tras reconciliar).	`CONSUMED`
`REJECTED`	Final. `PAYMENT_REJECTED`: pago rechazado, stock devuelto una vez. `OUT_OF_STOCK`: no había stock, sin reserva ni cobro.	`RELEASED / ninguna`

Los estados finales nunca se sobrescriben: todos los `UPDATE` de estado llevan `WHERE status IN ('PENDING_PAYMENT', 'PAYMENT_UNCERTAIN')`.

2. Casos de prueba: esperado vs. obtenido

Resultados obtenidos al ejecutar la colección completa contra los servicios en Docker (Collection Runner / `newman`): **29 requests, 99 assertions, 0 fallos**. Los mismos resultados se obtuvieron en ejecución local.

Caso	Qué se hizo	Resultado esperado	Resultado obtenido	
1. Flujo exitoso	Crear pedido del producto 1 con pago aprobado.	Un pedido, una reserva, un cobro y estado final correcto.	**201** · `PAID` · reserva `CONSUMED` · 1 cobro `APPROVED` · stock 5 → 4.	✓
2. Misma solicitud repetida	Reenviar dos veces el mismo pedido con la misma clave; luego, misma clave con otro cuerpo.	Se reconoce la operación previa; no se duplica pedido, reserva ni cobro.	**201** + `Idempotent-Replayed: true` con el mismo `id` · stock sigue en 4 · sigue habiendo 1 cobro. Con otro cuerpo: **422** `idempotency-key-reused`, indicando el pedido existente.	✓
3. Respuesta perdida después del cobro	Crear pedido con `payment_mode: timeout`; consultar el mock; reintentar con la misma clave.	El backend resuelve el estado consultando el pago y no vuelve a cobrar a ciegas.	Primer intento: **202** `PAYMENT_UNCERTAIN` a los 2,07 s, reserva intacta. El mock ya tenía el cobro `APPROVED`. Reintento: **201** `PAID` con `reconciled_at` registrado · **1 solo cobro** · stock 4 → 3.	✓
4. Pago rechazado	Crear pedido con `payment_mode: reject`; reenviarlo con la misma clave.	El pedido queda rechazado y la reserva se libera una sola vez.	**402** `REJECTED` / `PAYMENT_REJECTED` · reserva `RELEASED` · stock vuelve a 3. El reenvío responde 402 (replay) y el stock **sigue en 3**.	✓
5. Otra intención de compra	Producto 2 con 1 unidad: compra con clave A y luego con clave B.	La nueva solicitud es una operación distinta y el stock nunca queda negativo.	A: **201** `PAID`. B: **409** `out-of-stock`, pedido nuevo `REJECTED`/`OUT_OF_STOCK`, sin reserva y sin cobro (404 en el mock) · stock = **0**.	✓
Persistencia	`docker compose restart` de ambos servicios tras la corrida.	Los estados siguen consultables después de reiniciar.	5 pedidos antes y después; 4 pagos en el mock; el pedido del caso 3 sigue `PAID` con su `reconciled_at`.	✓
Extra: concurrencia	10 solicitudes simultáneas: (a) con claves distintas por la última unidad; (b) con la misma clave.	(a) Solo una compra. (b) Un solo pedido y un solo cobro.	(a) 1×201 y 9×409, stock 0, 1 cobro. (b) 10×201 con el mismo pedido, 1 cobro. El caso (a) se verificó también en Docker con gunicorn (2 procesos × 4 hilos).	✓
Extra: mock caído	Detener el mock, crear pedido, reconciliar; levantar el mock y reconciliar.	Sin respuesta no se decide nada; al volver se resuelve sin duplicar.	Sigue `PAYMENT_UNCERTAIN` mientras el mock está caído. Al volver: el mock responde 404, se cobra con la misma clave y el pedido pasa a `PAID`.	✓

3. Capturas de Postman

Capturas de la ejecución manual en Postman, caso por caso (claves fijas `caso1-pedido`, `caso3-pedido`, etc.). Cada una muestra el método, la URL, el código de respuesta y el cuerpo devuelto.

Caso 1: flujo exitoso

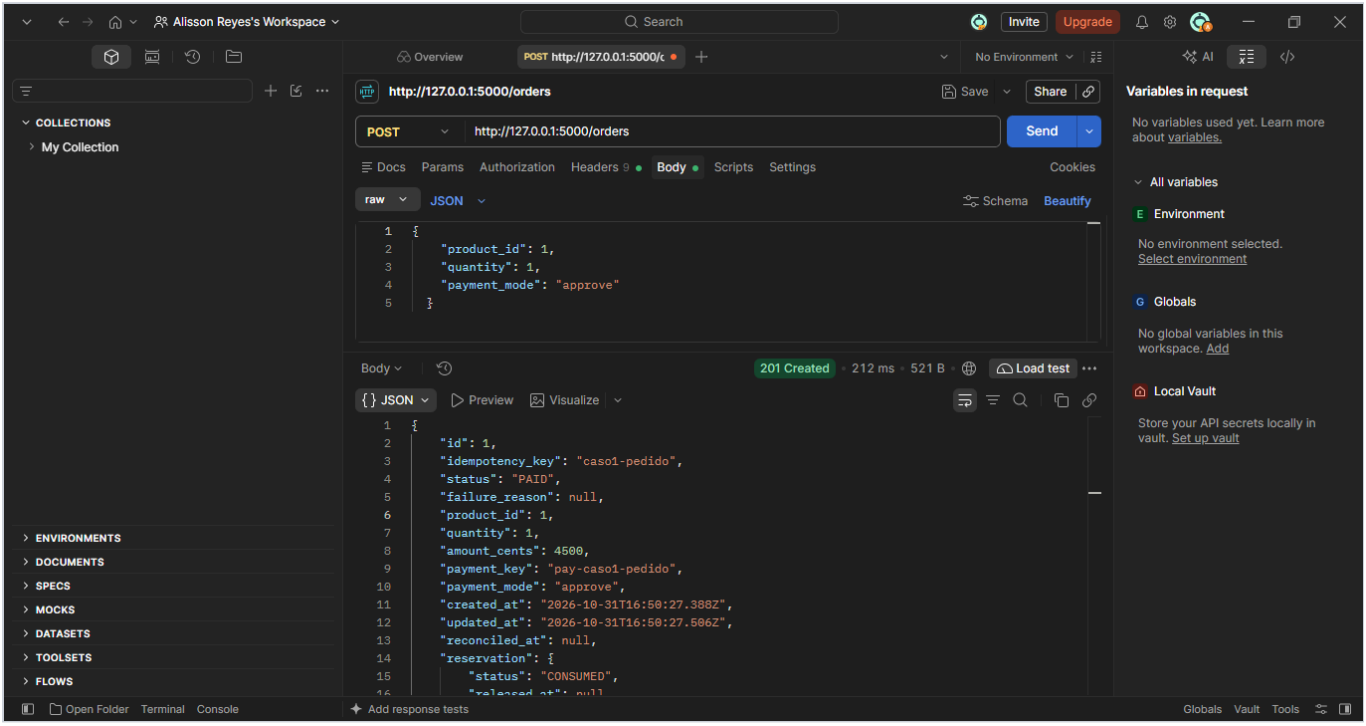


Figura 1. Caso 1 · Paso 1: Crear pedido con pago aprobado

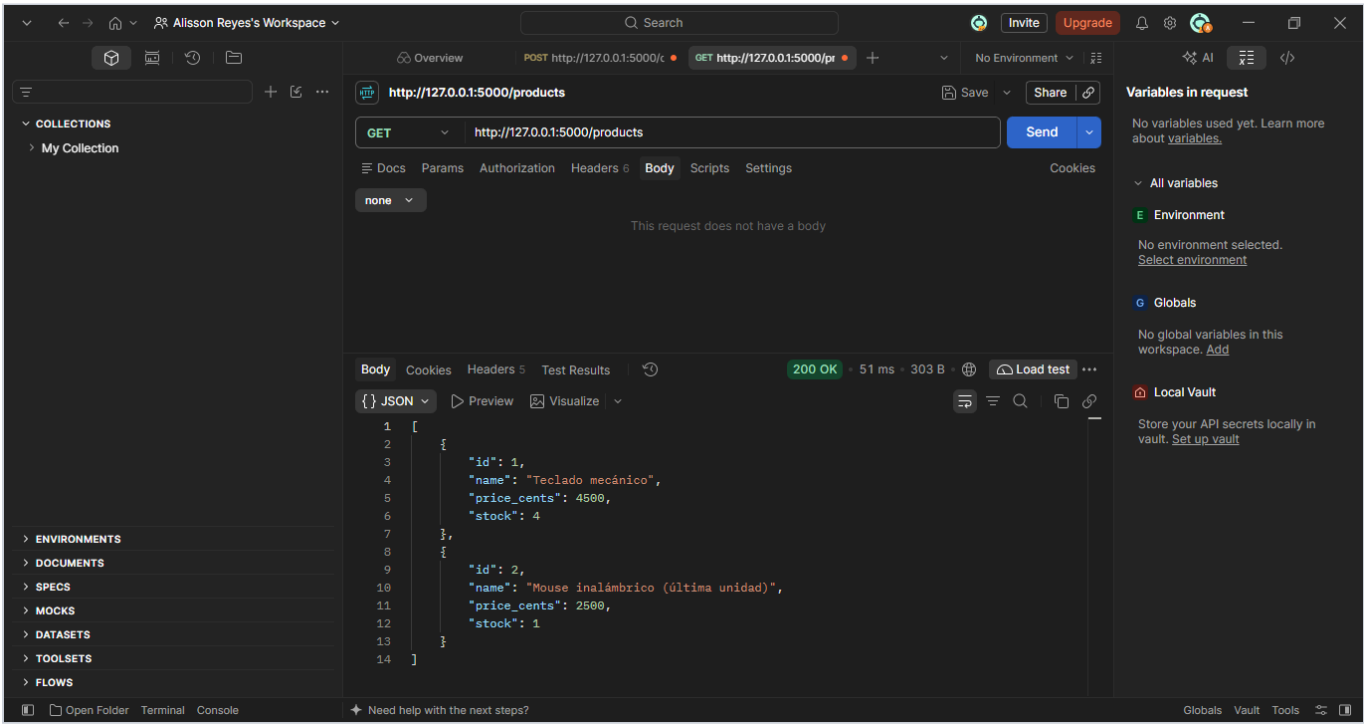


Figura 2. Caso 1 · Paso 2: Verificación de reducción de stock al realizar un pedido

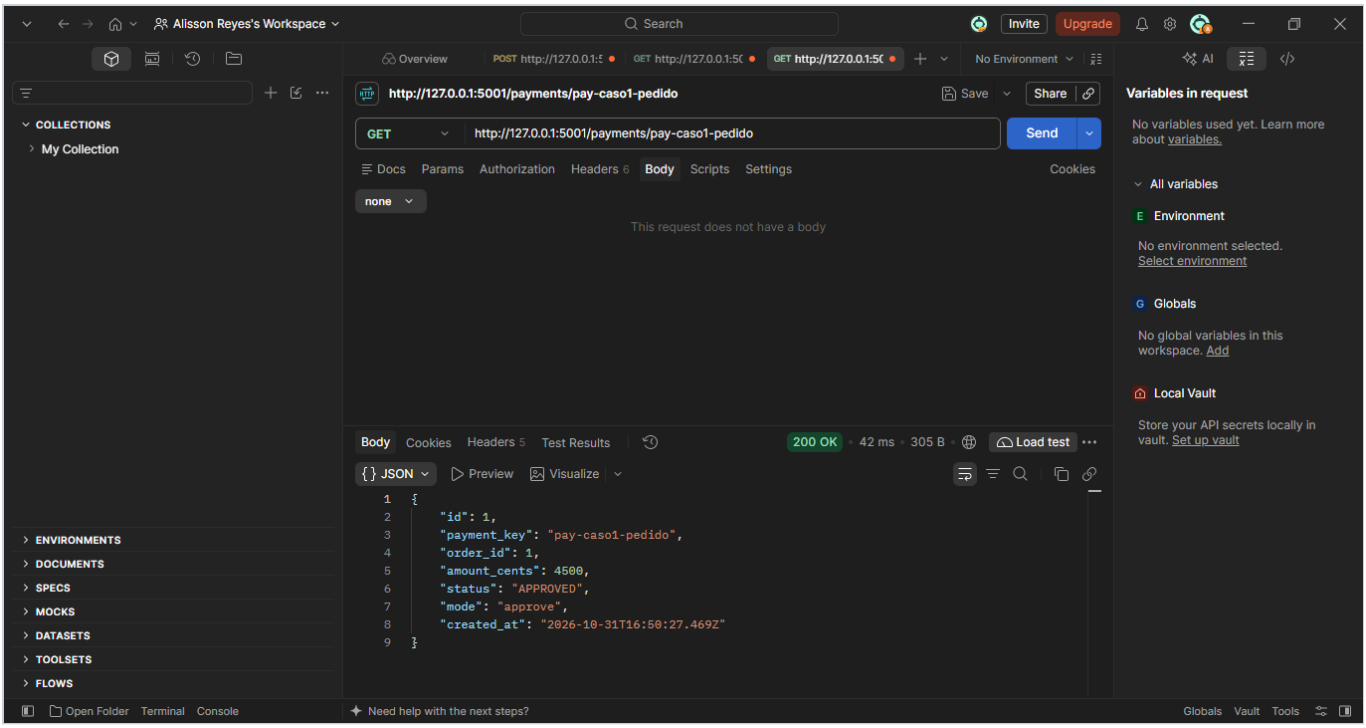


Figura 3. Caso 1 · Paso 3: Verificación de cobro

Caso 2: misma solicitud repetida

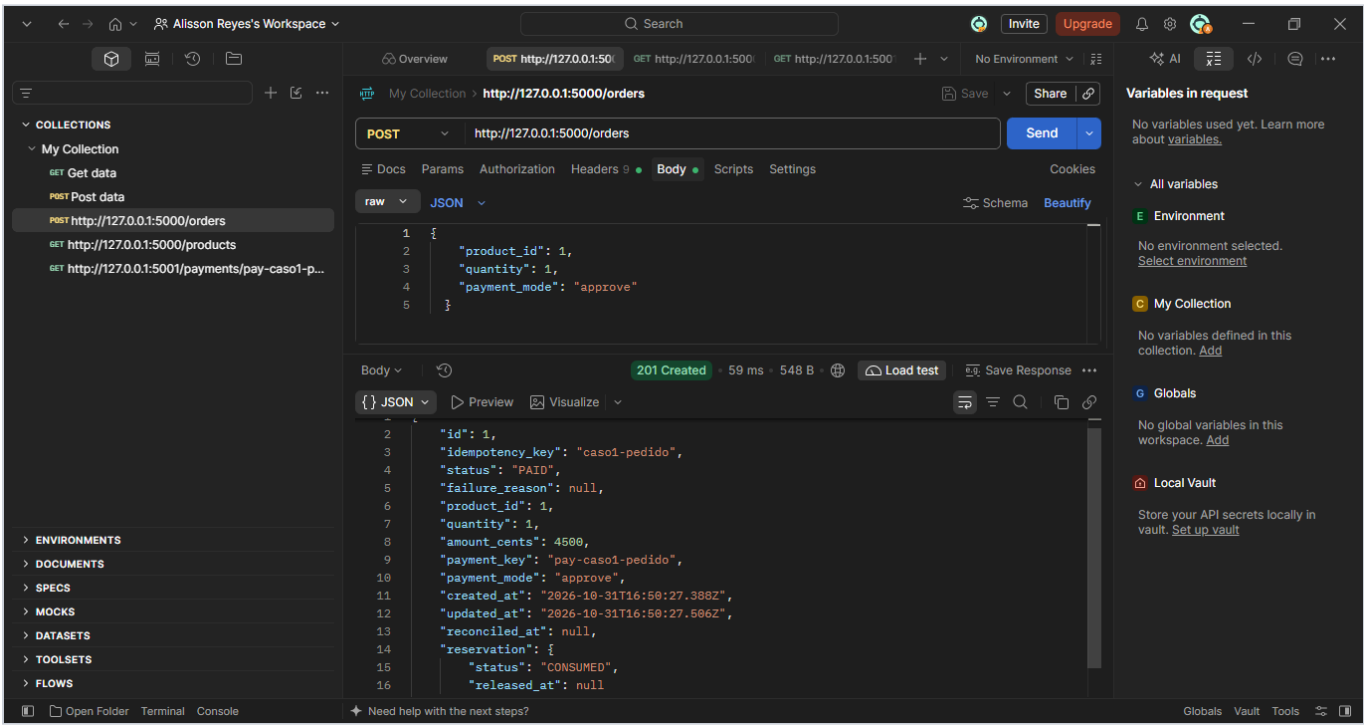


Figura 4. Caso 2 · Paso 1: Reenvío del pedido del caso 1

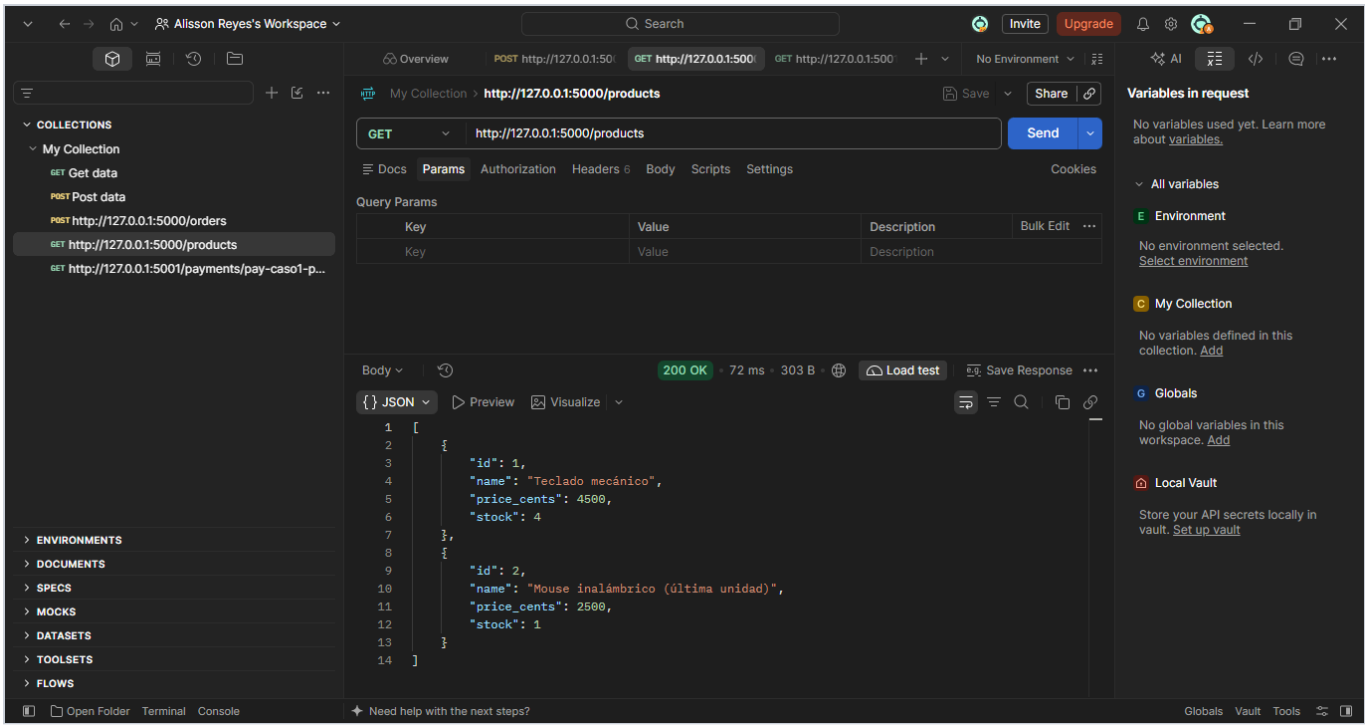


Figura 5. Caso 2 · Paso 2: Verificación de que el stock no cambió con el reenvío

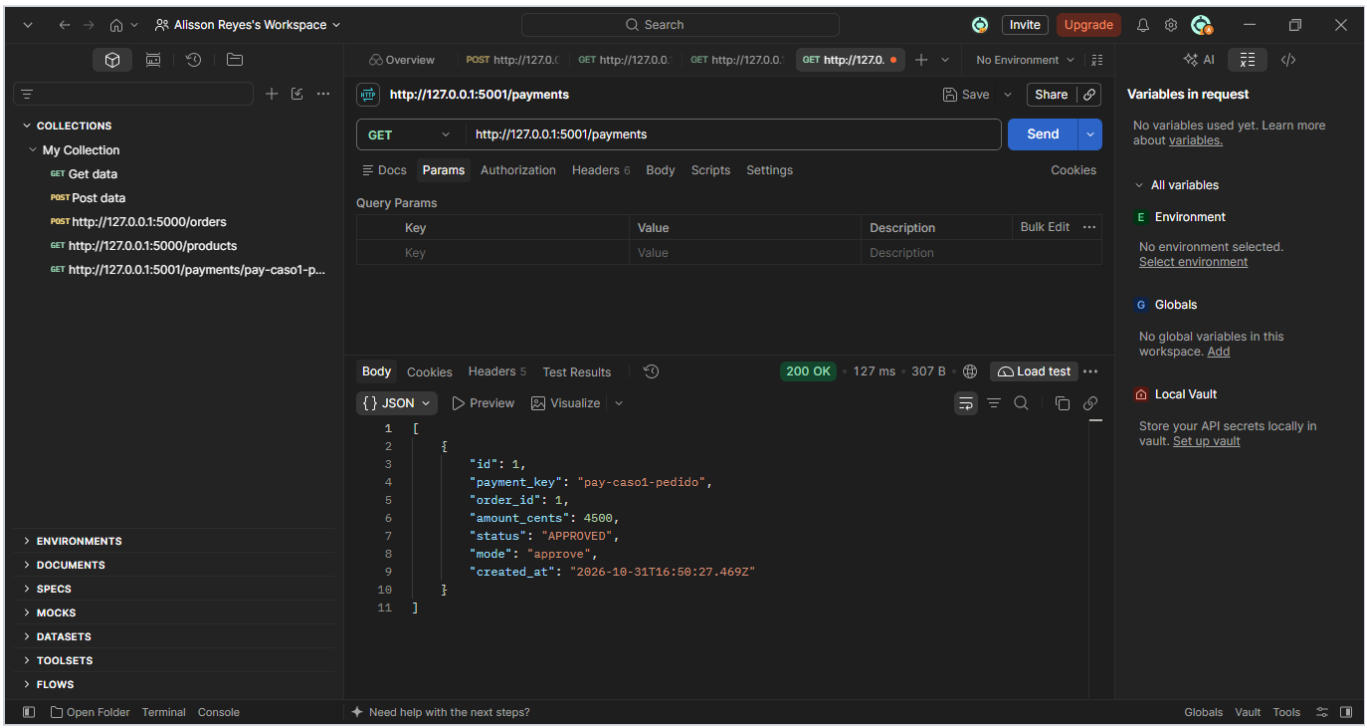


Figura 6. Caso 2 · Paso 3: Verificación de que sigue existiendo un solo cobro

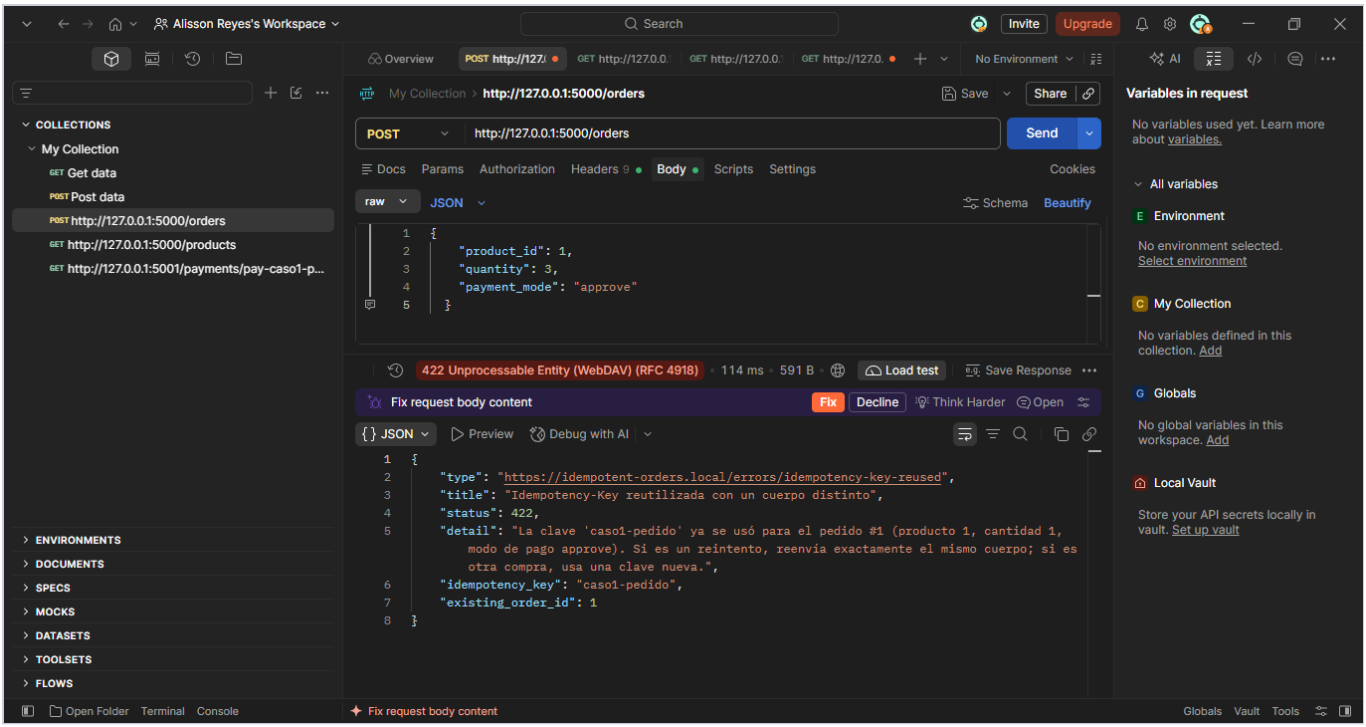


Figura 7. Caso 2 · Paso 4: Verificación de que la misma clave no se pueda usar para un pedido distinto

Caso 3: respuesta perdida después del cobro

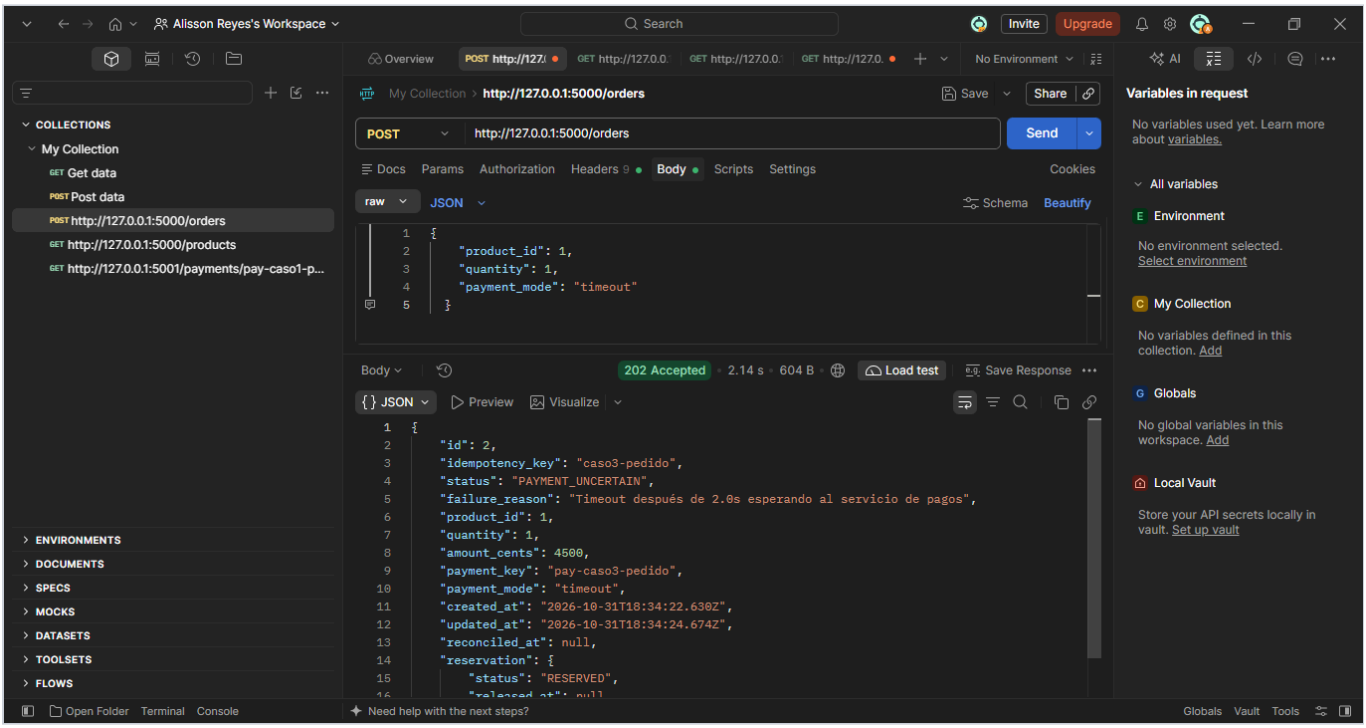


Figura 8. Caso 3 · Paso 1: Crear pedido en modo timeout

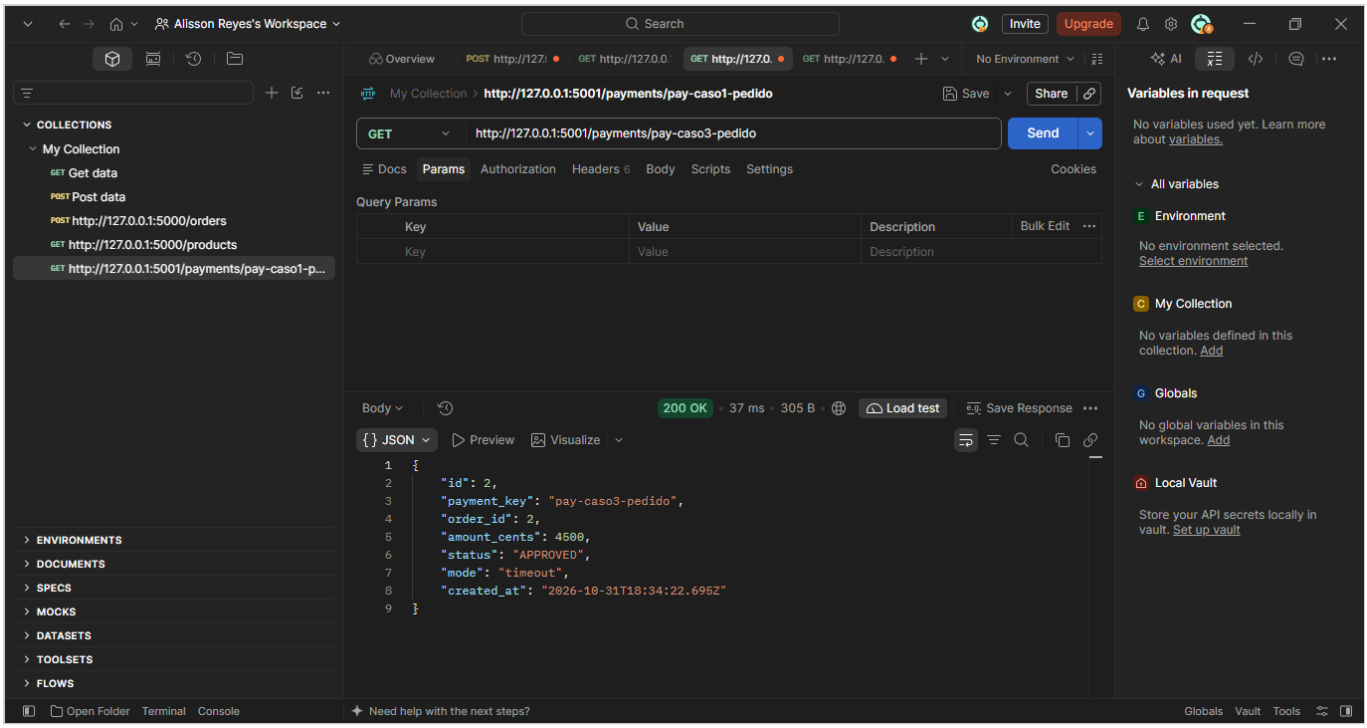


Figura 9. Caso 3 · Paso 2: Comprobar el registro del cobro

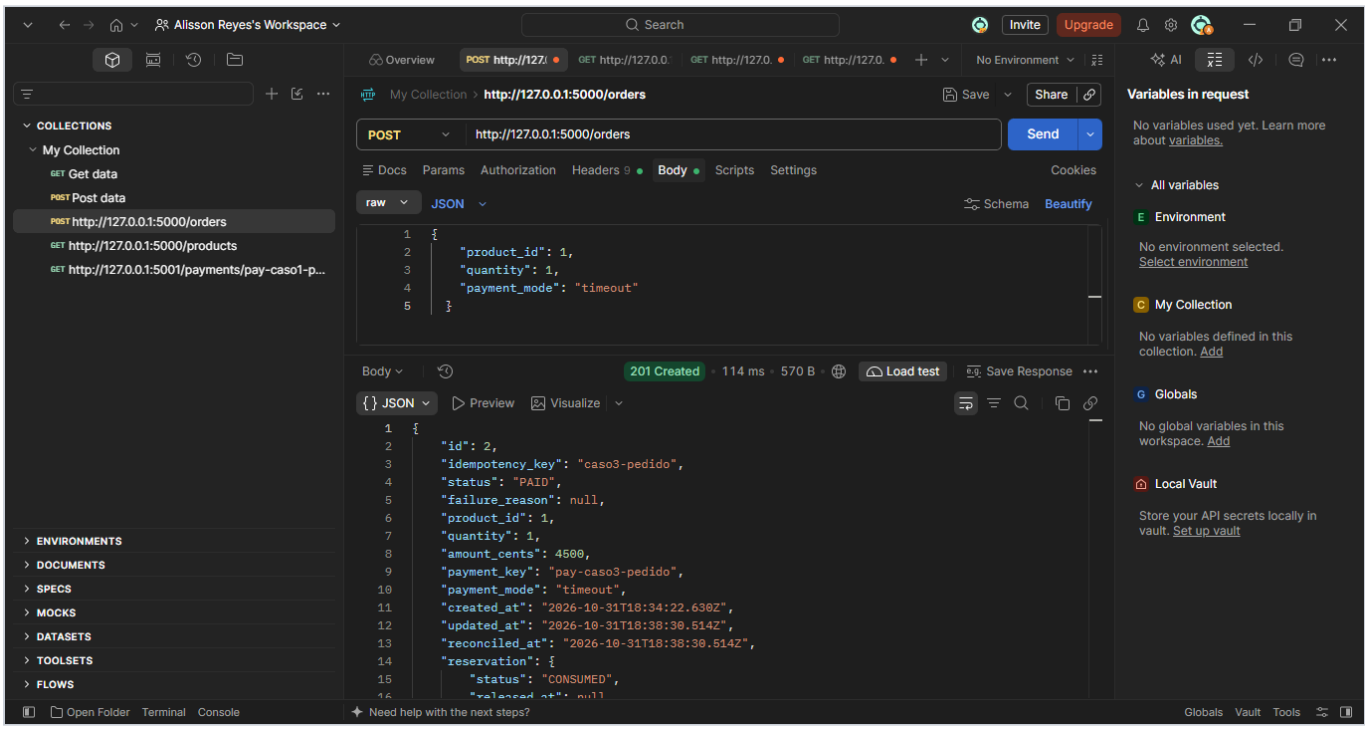


Figura 10. Caso 3 · Paso 3: Reintento de pedido con la misma clave

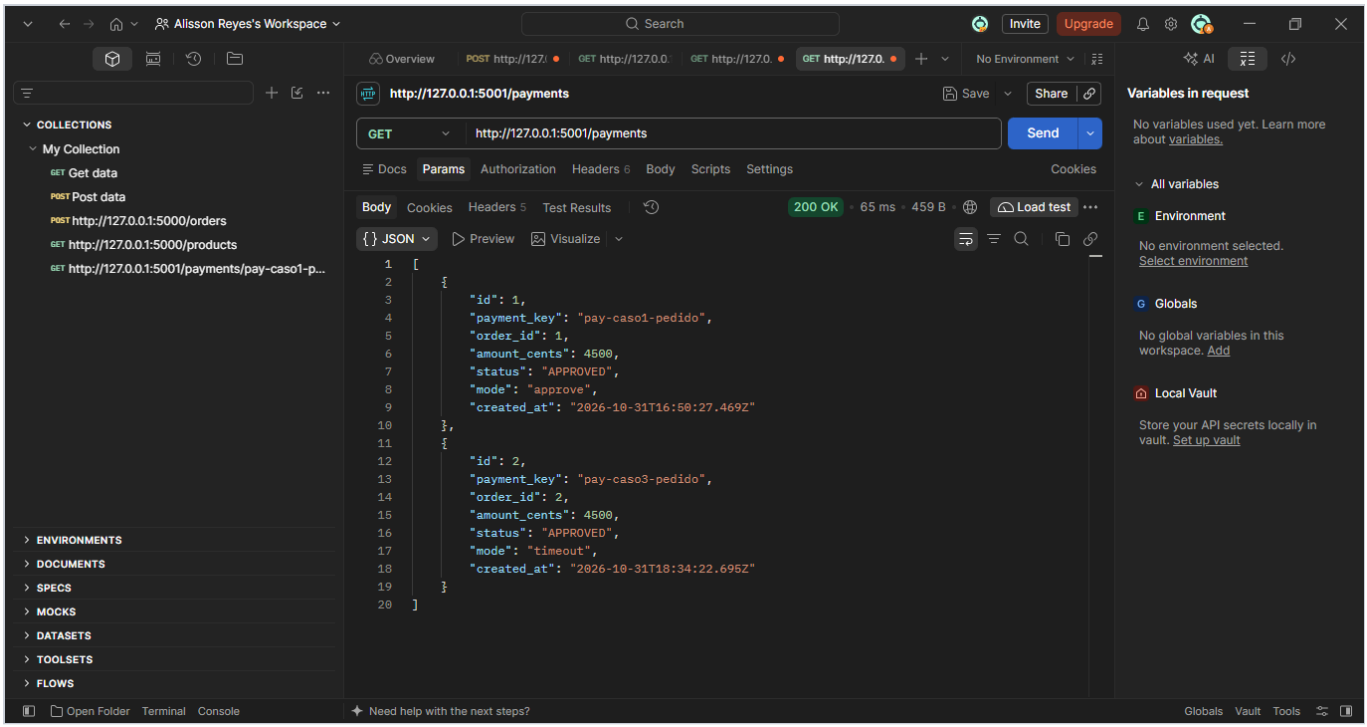


Figura 11. Caso 3 · Paso 4: Verificar existencia de un solo cobro para el pedido 3

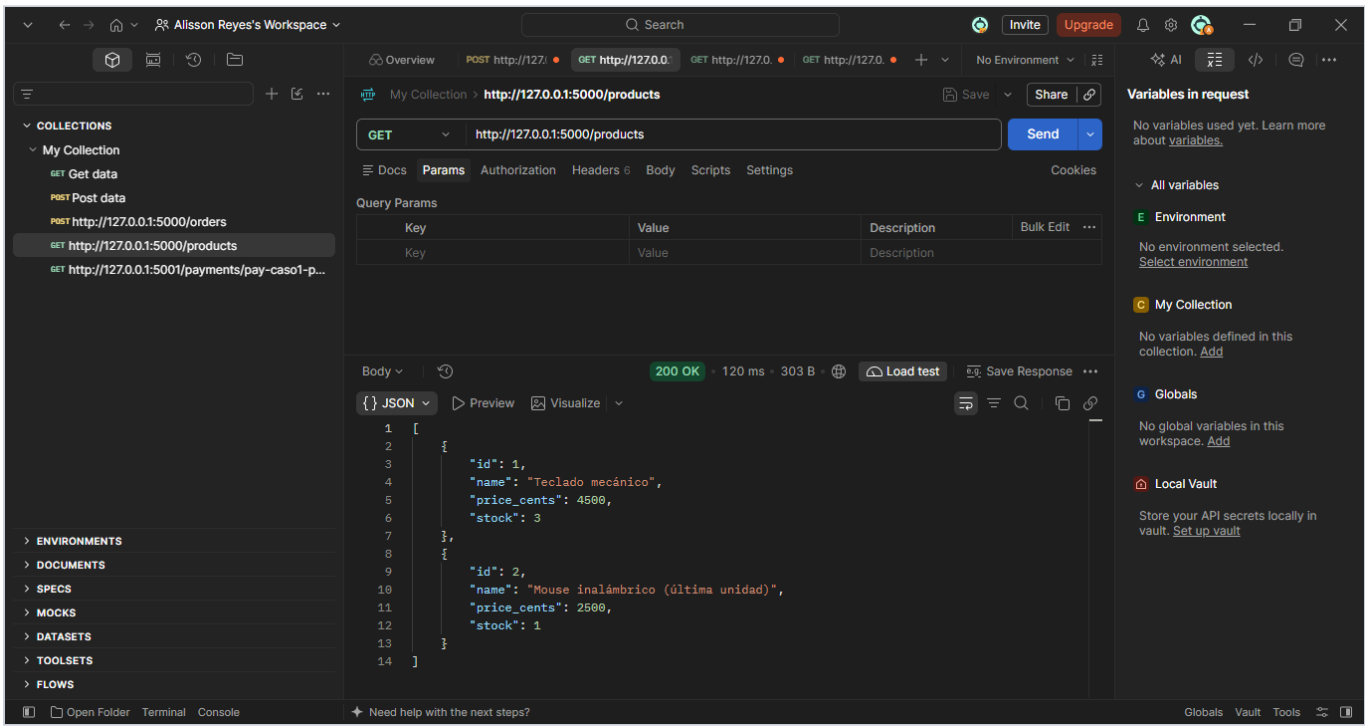


Figura 12. Caso 3 · Paso 5: Verificación de reducción de stock al realizar el pedido 3

Caso 4: pago rechazado

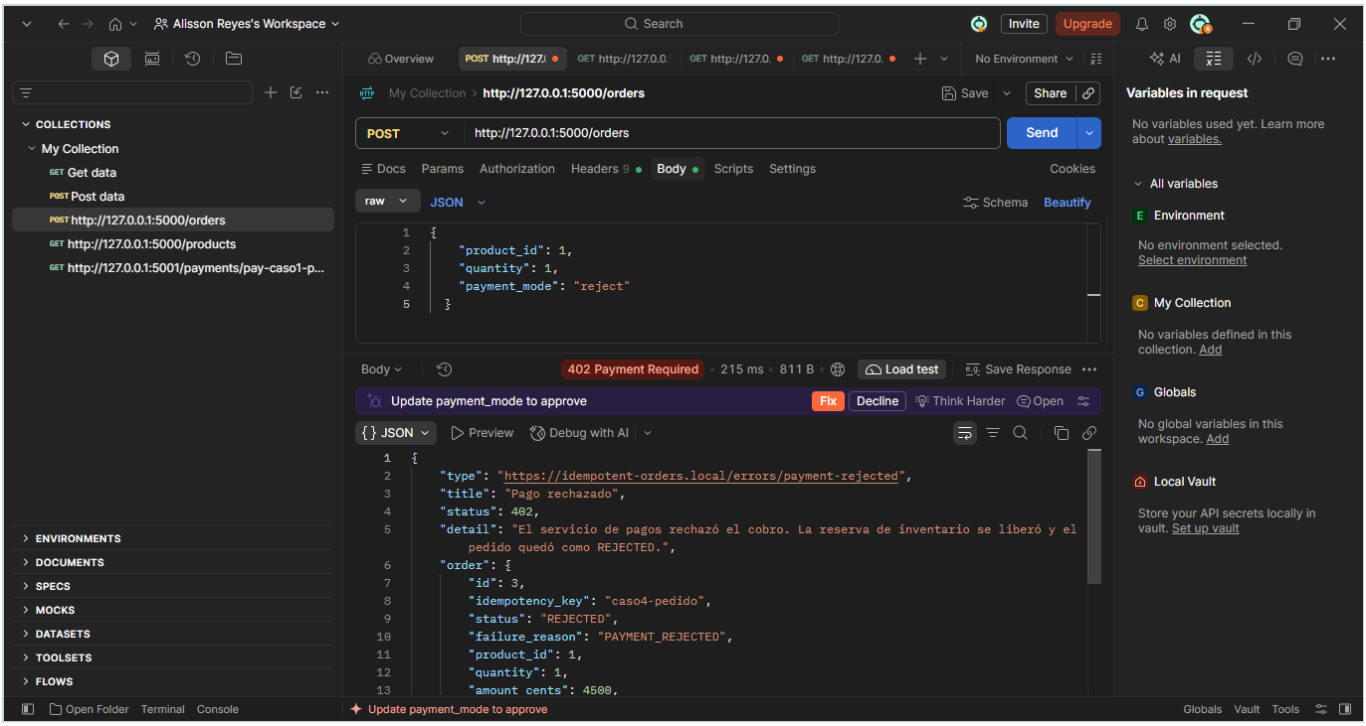


Figura 13. Caso 4 · Paso 1: Crear pedido con pago rechazado

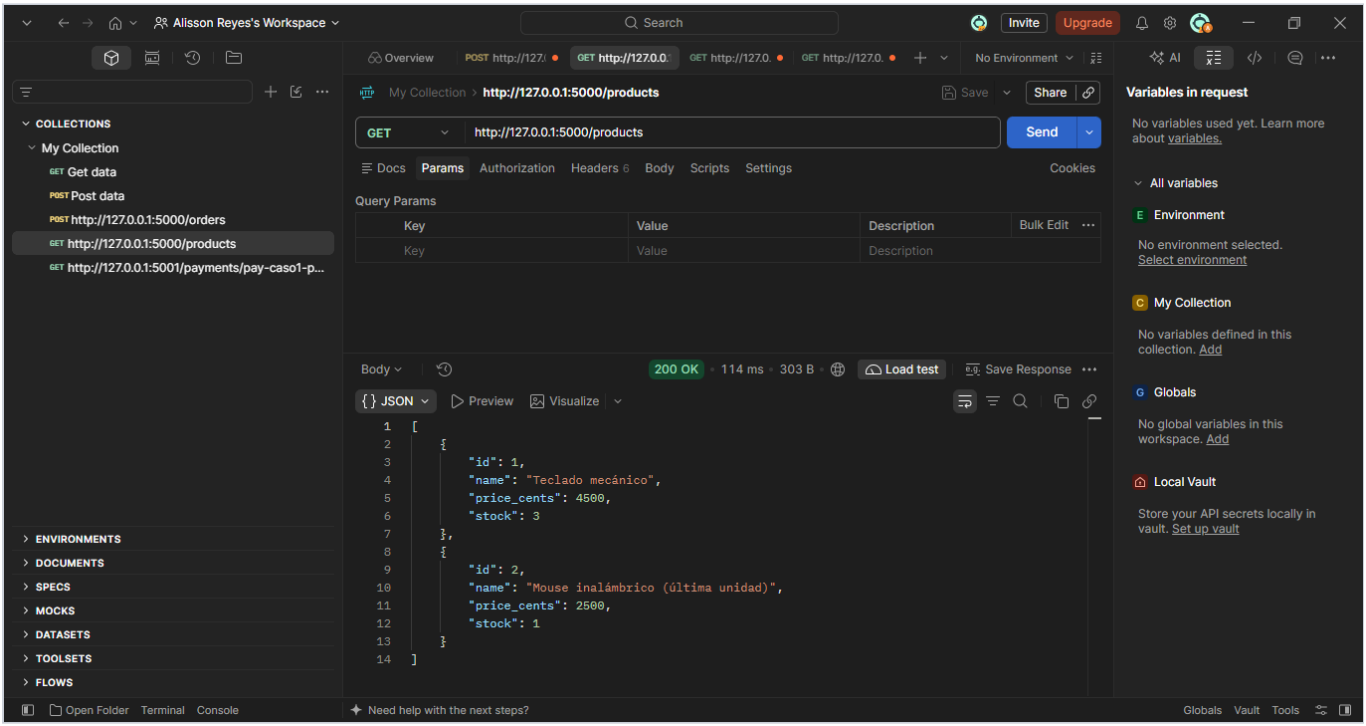


Figura 14. Caso 4 · Paso 2: Verificar que el stock volvió debido a que el pago se rechazó

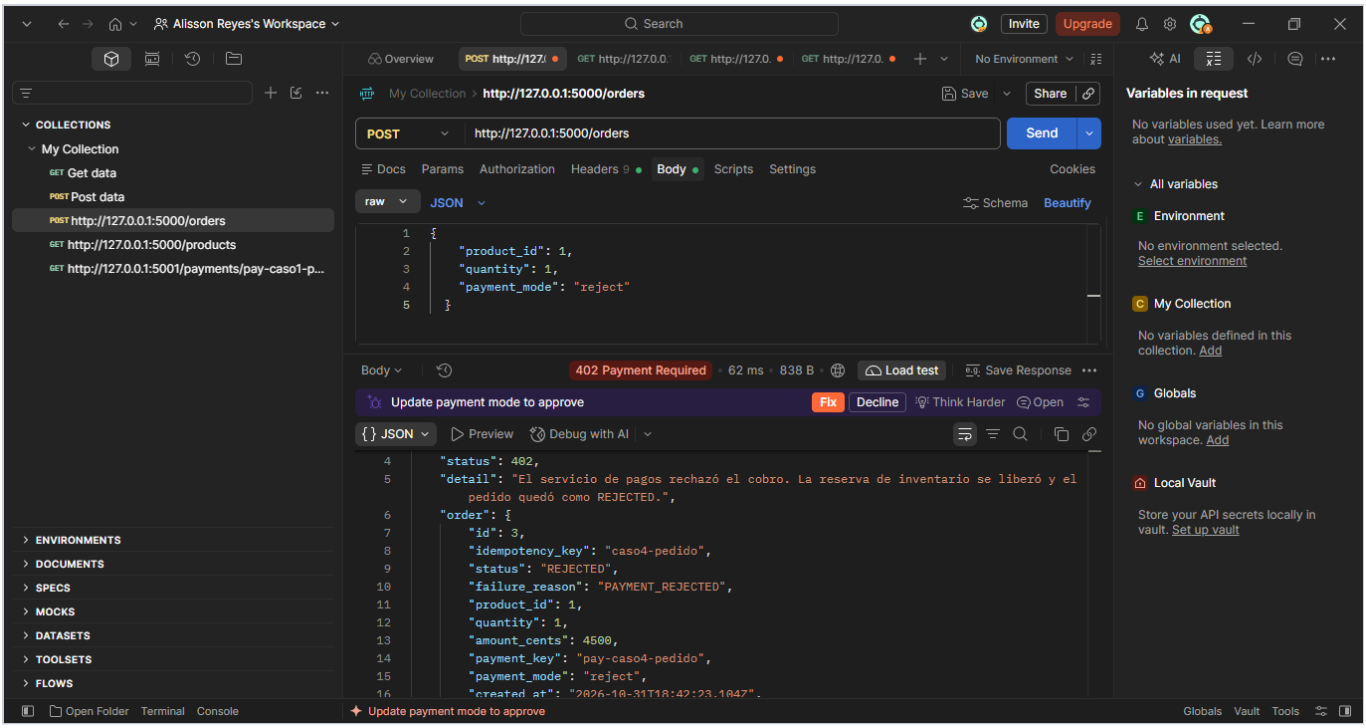


Figura 15. Caso 4 · Paso 3: Reenviar el pedido rechazado

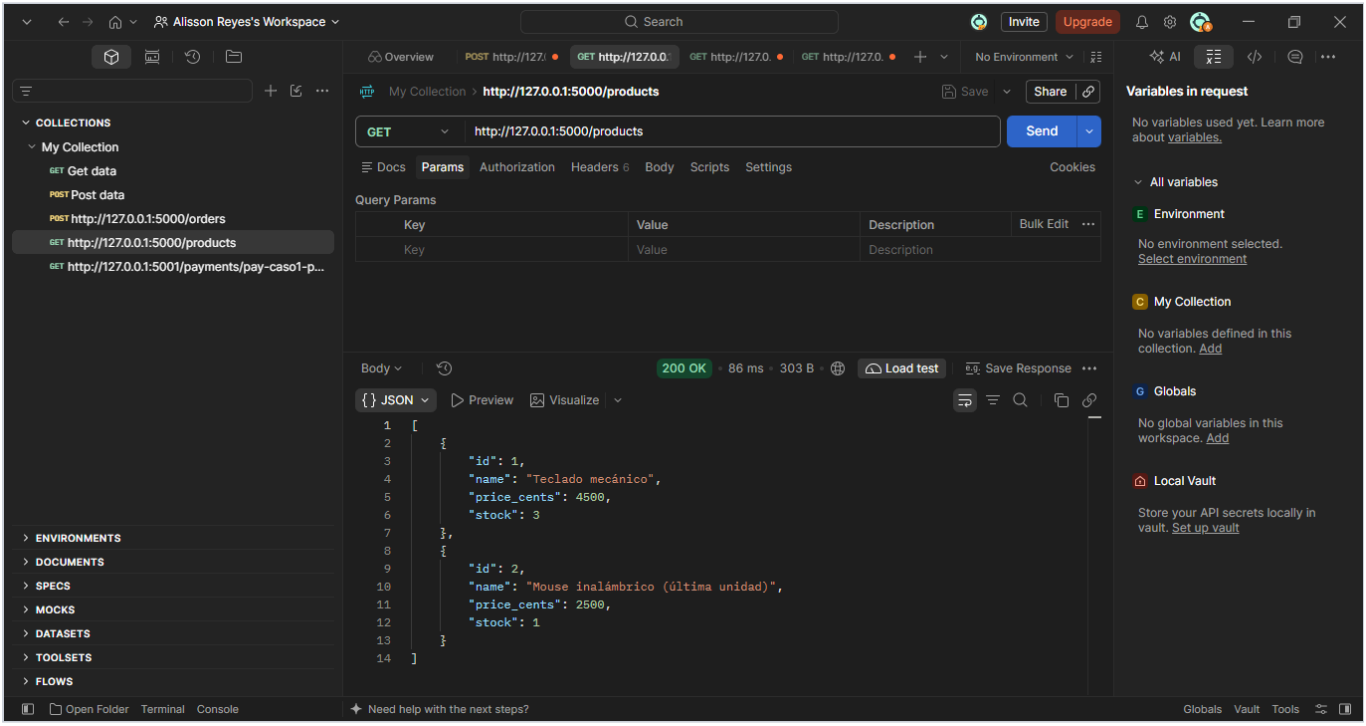


Figura 16. Caso 4 · Paso 4: Verificar que el stock NO se devolvió dos veces tras el reenvío del pedido 4

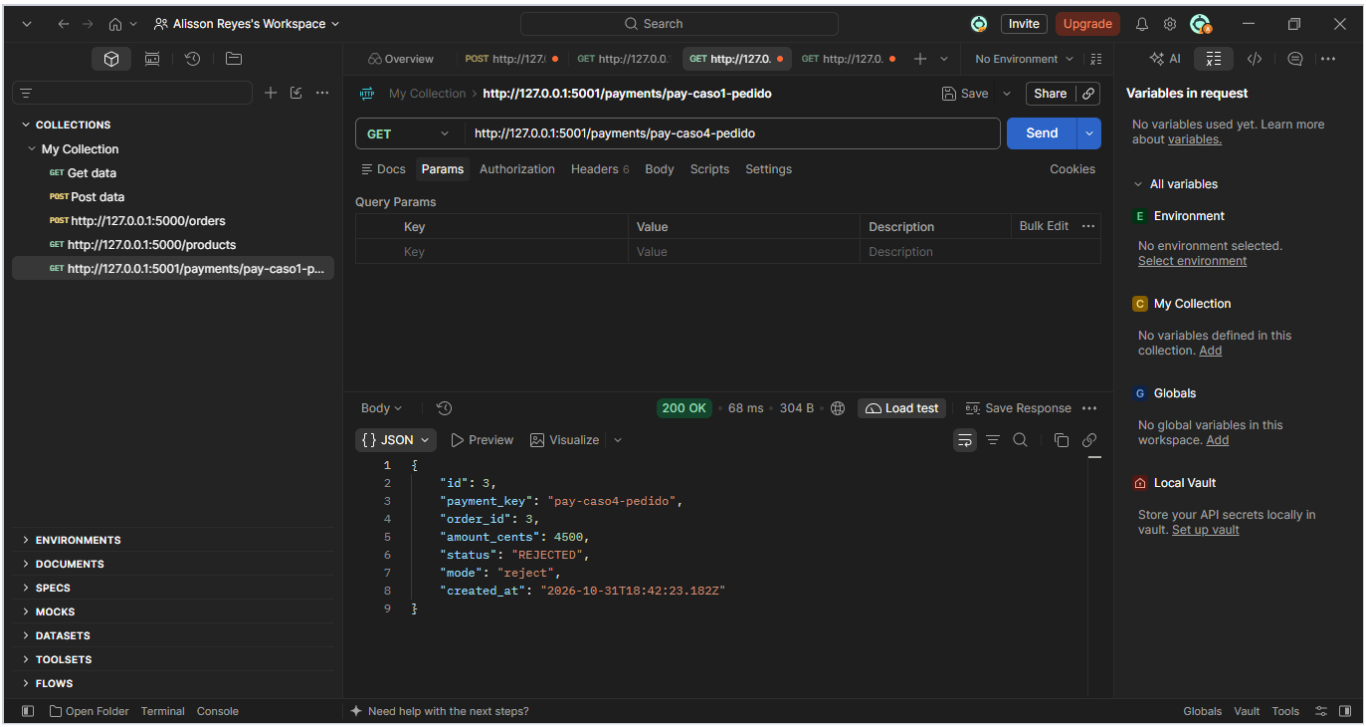


Figura 17. Caso 4 · Paso 5: Verificación del cobro rechazado

Caso 5: otra intención de compra con la última unidad

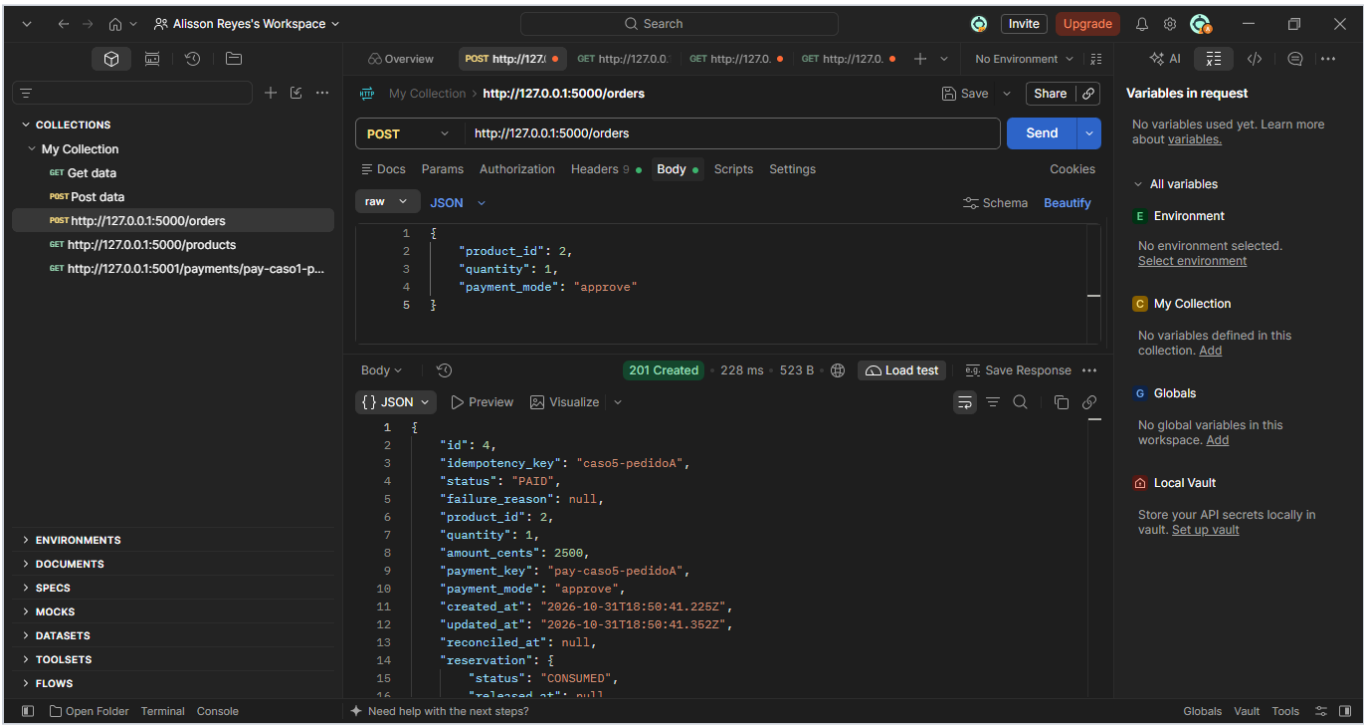


Figura 18. Caso 5 · Paso 1: Creación de pedido para comprar la última unidad (clave A)

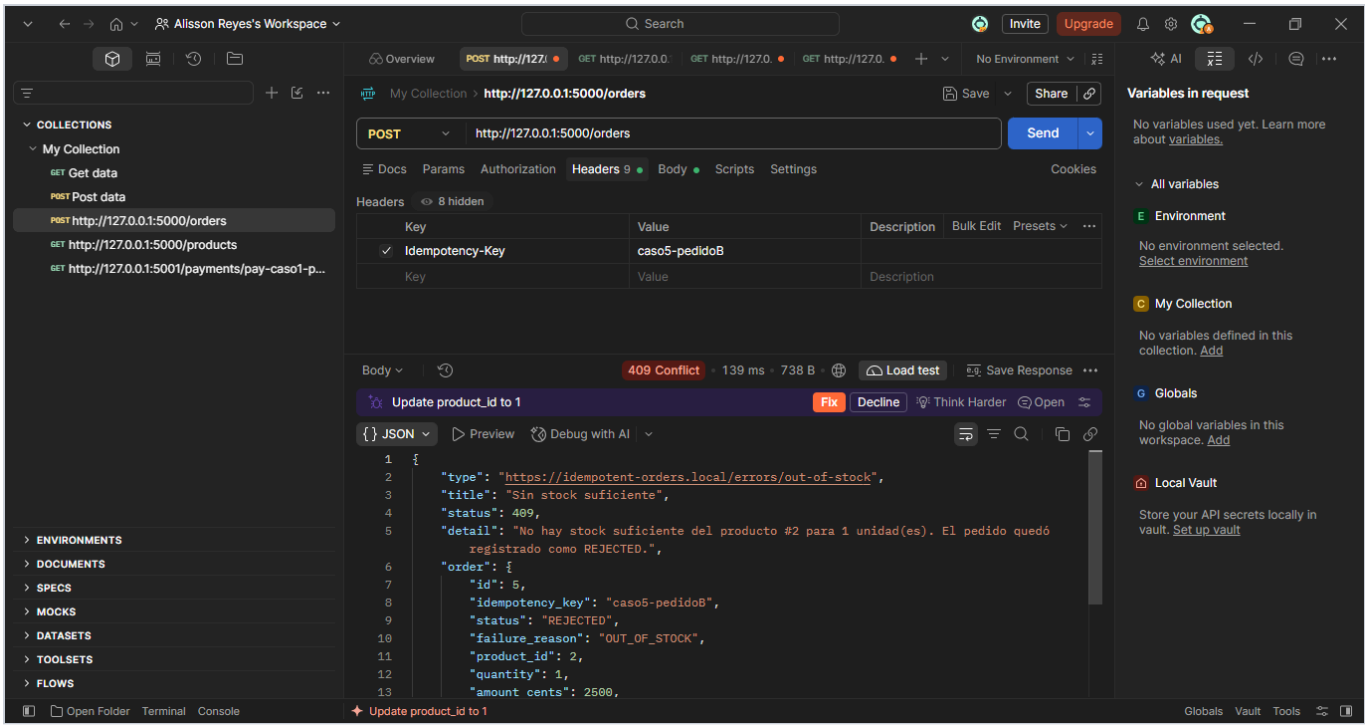


Figura 19. Caso 5 · Paso 2: Intento de comprar otra vez con otra clave (clave B) y stock insuficiente

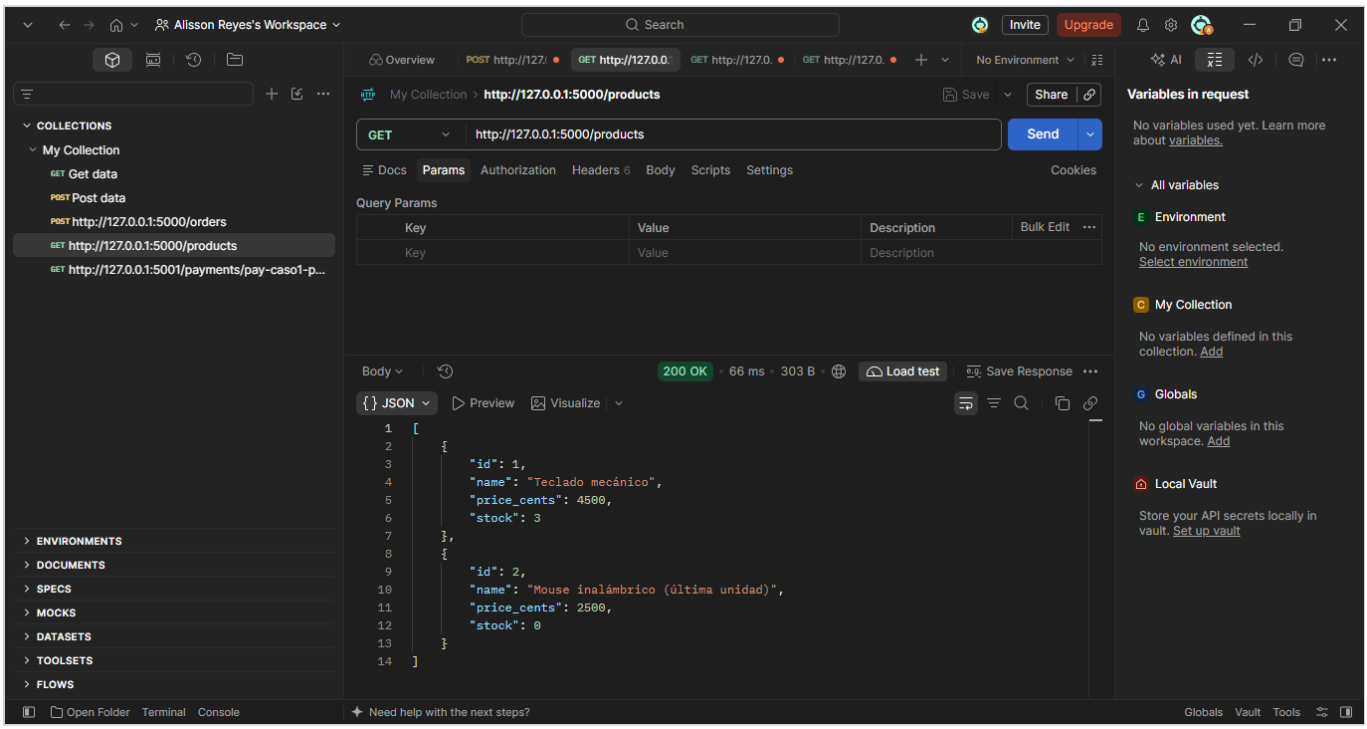


Figura 20. Caso 5 · Paso 3: Verificar que el stock está en 0 y que no es un número negativo

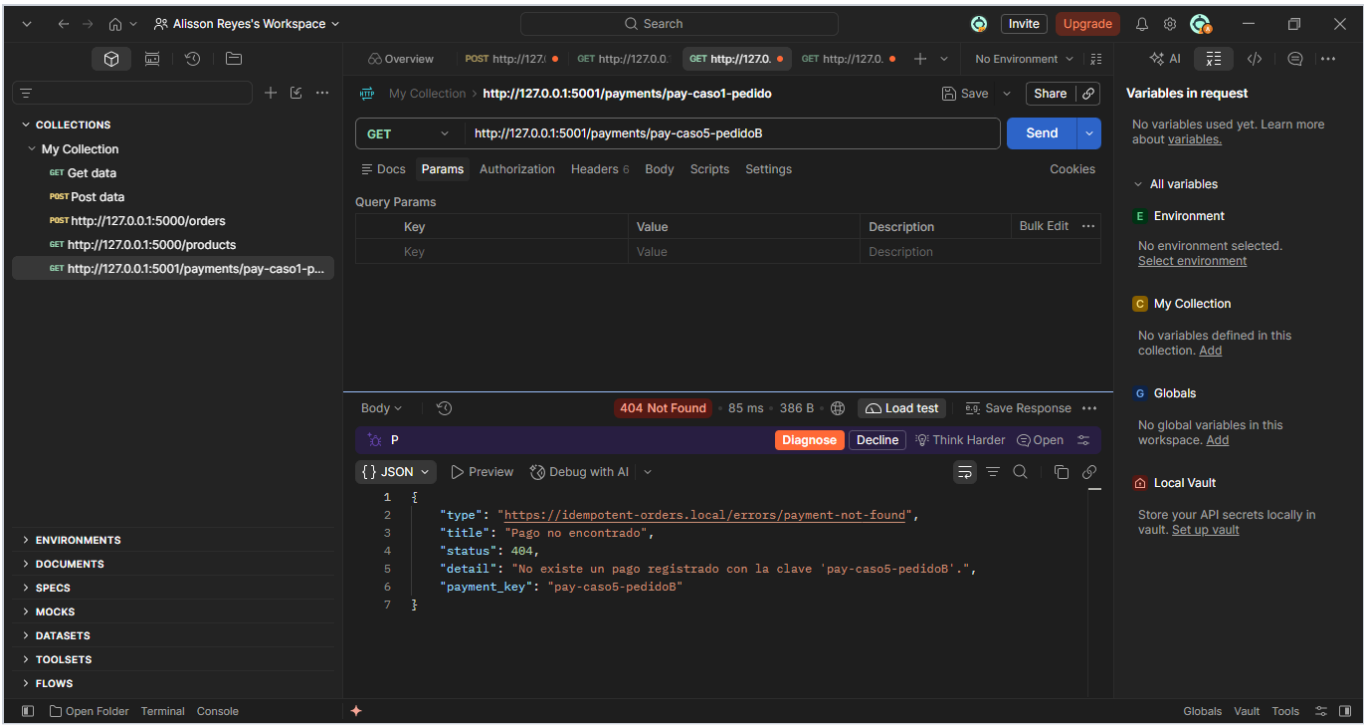


Figura 21. Caso 5 · Paso 4: Verificar que no se intentó cobrar al realizar el pedido 5 con clave B

Verificación final

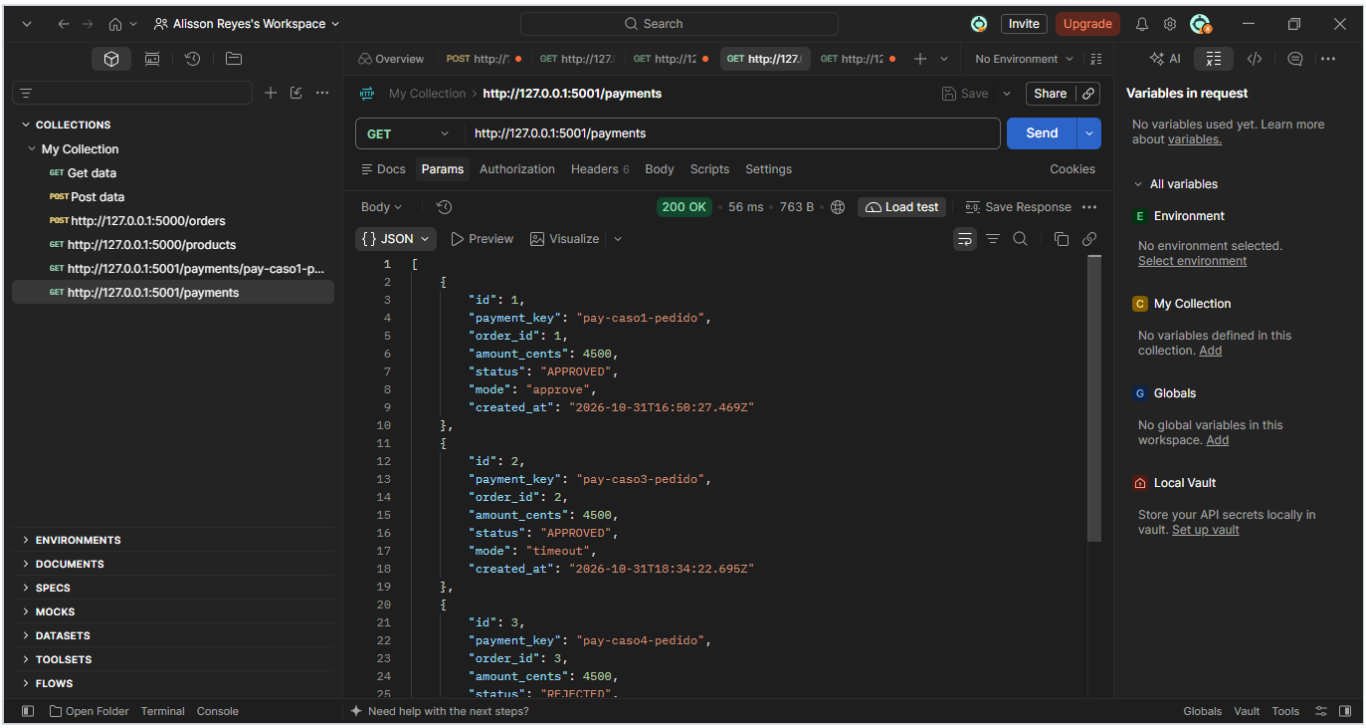


Figura 22. Verificación final: Visualización de todos los cobros captura 1

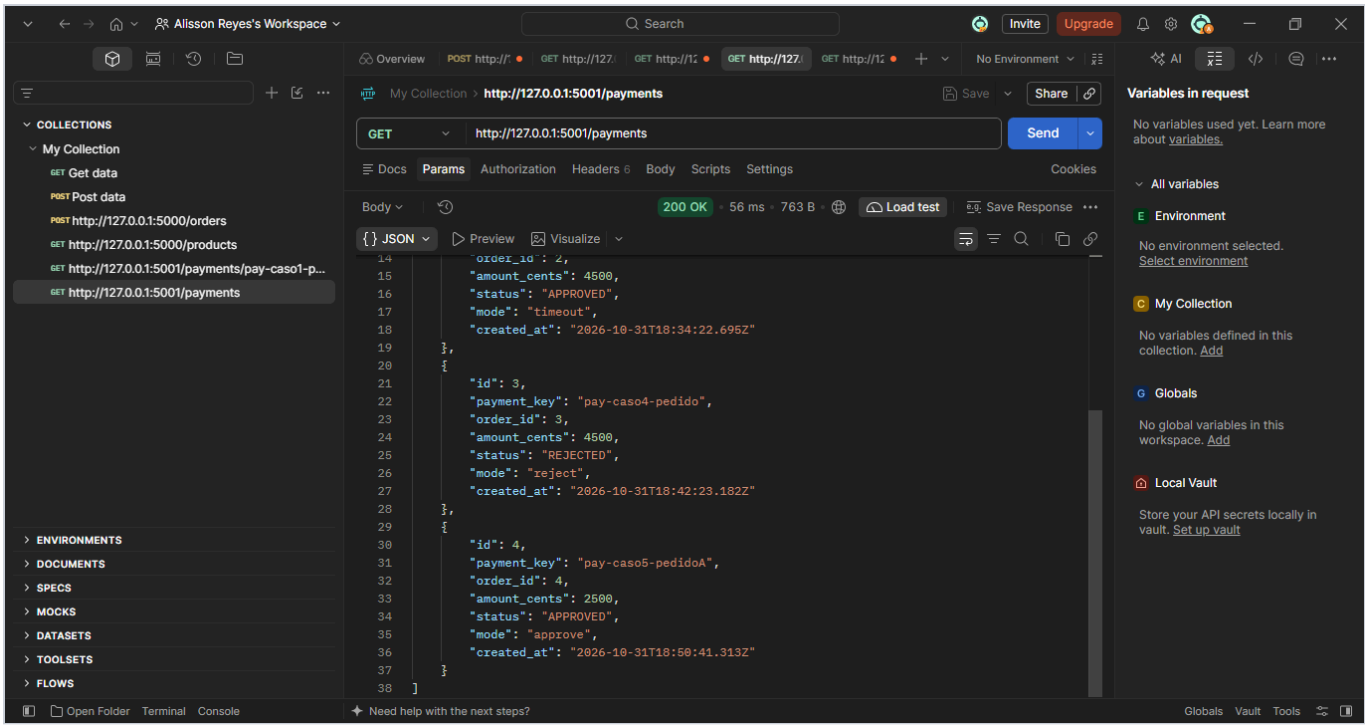


Figura 23. Verificación final: Visualización de todos los cobros captura 2

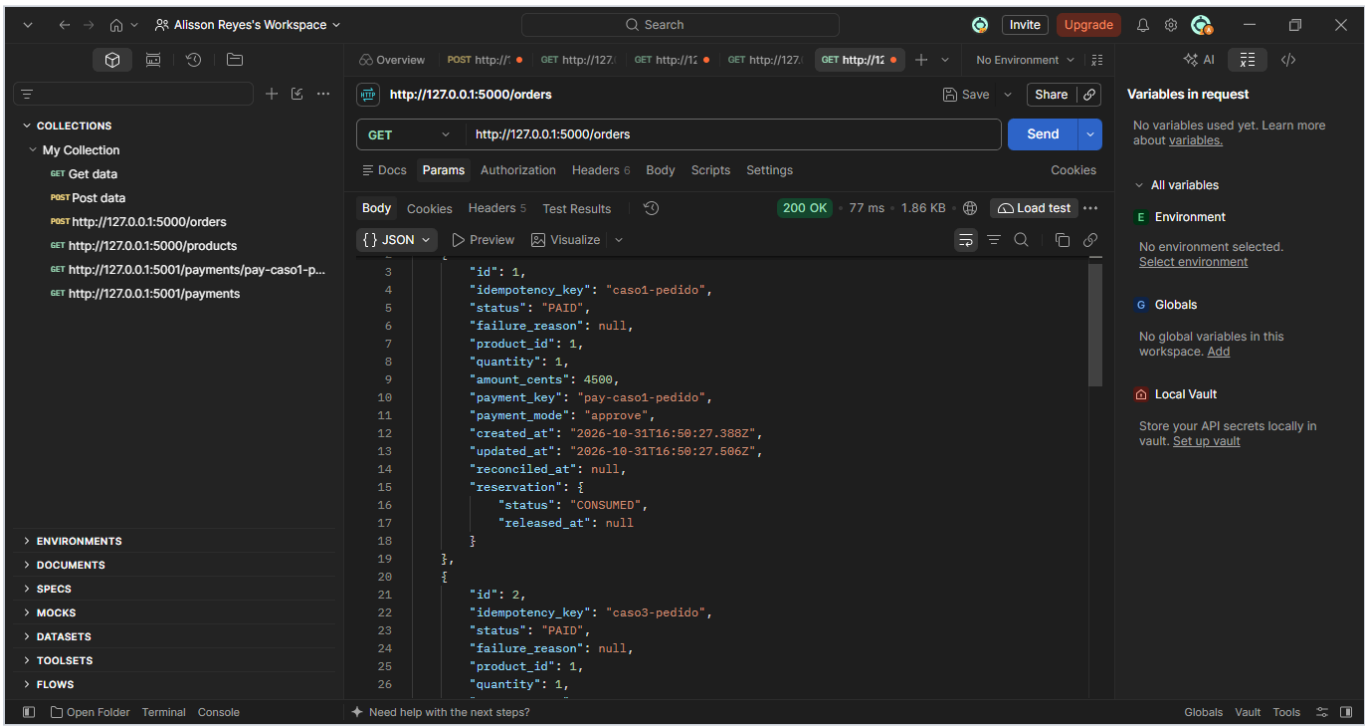


Figura 24. Verificación final: Visualización de todos los pedidos captura 1

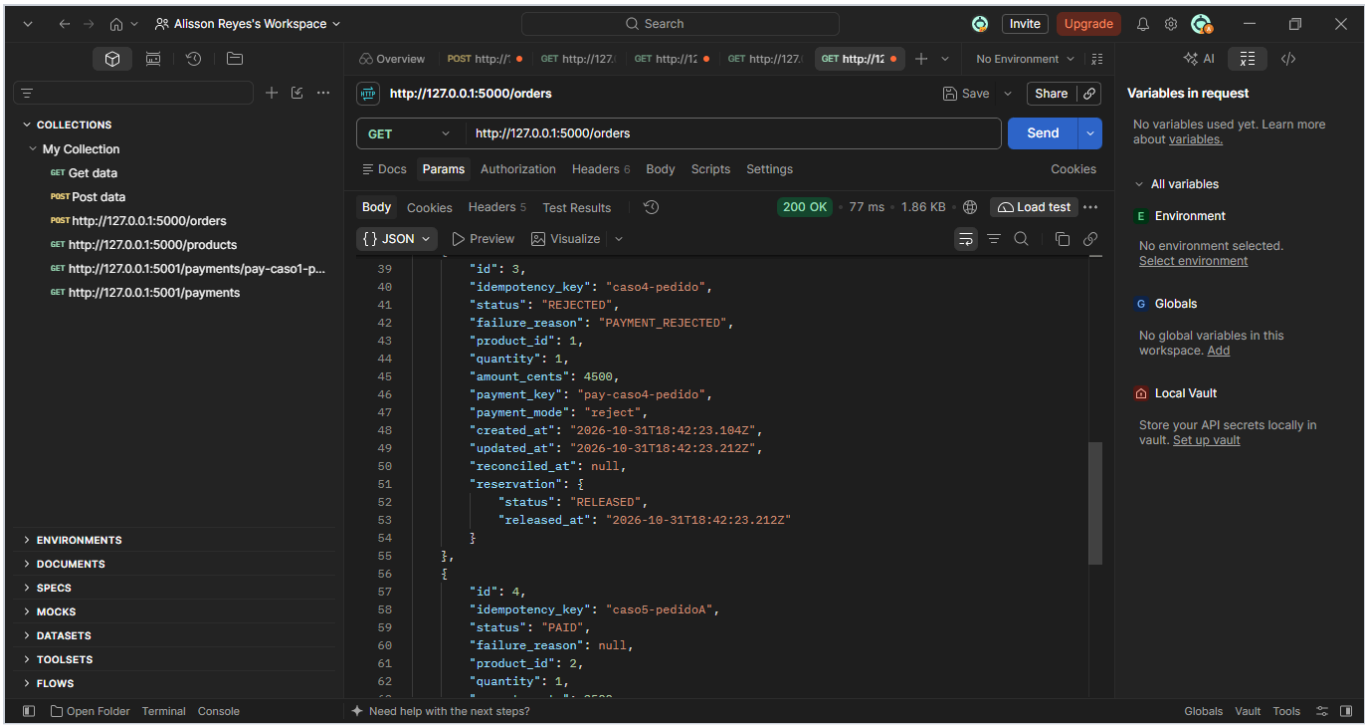


Figura 25. Verificación final: Visualización de todos los pedidos captura 2

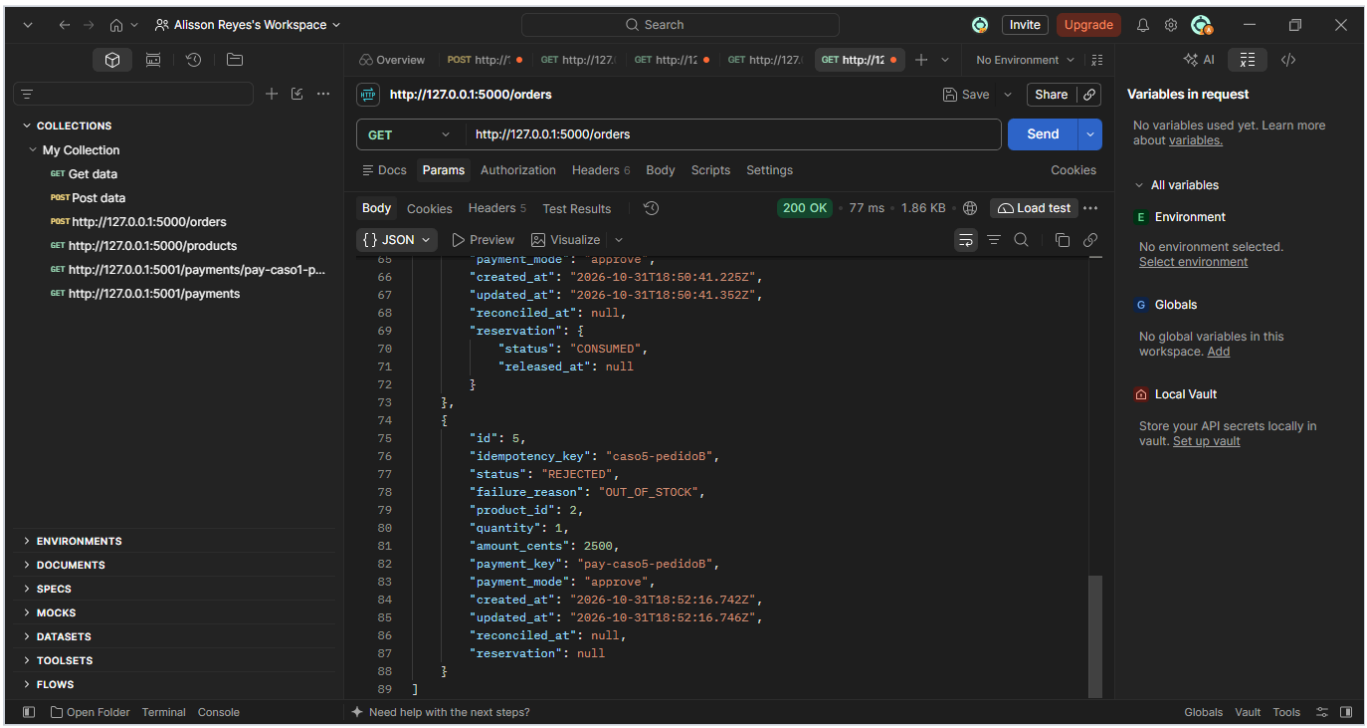


Figura 26. Verificación final: Visualización de todos los pedidos captura 3

4. Evidencia en SQLite

Las bases revisadas son las de la ejecución manual de la sección 3 (claves `caso1-pedido`, `caso3-pedido`, `caso4-pedido`, `caso5-pedidoA` y `caso5-pedidoB`). Se abrieron en DBeaver en modo solo lectura: `orders.db` como conexión principal y `payments.db` adjunta como `pay`.

Tablas de orders.db

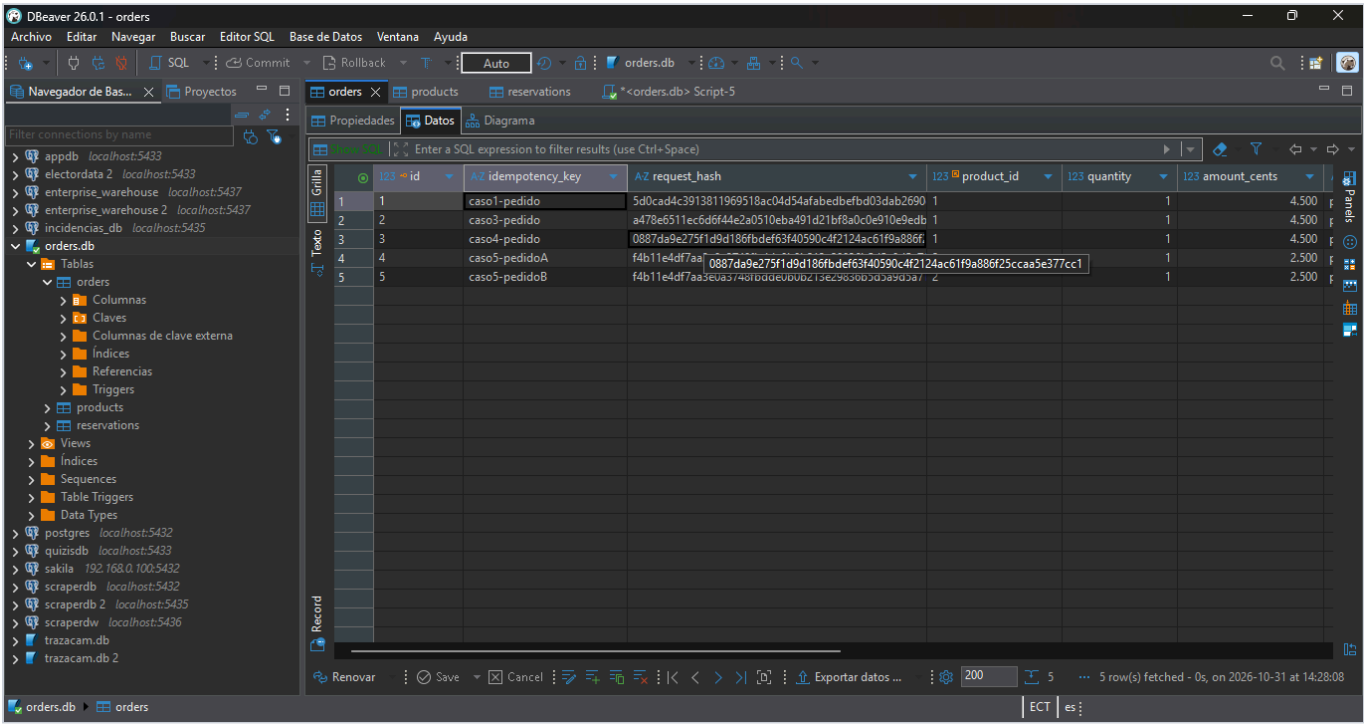


Figura 27. Tabla orders: 5 pedidos, uno por Idempotency-Key

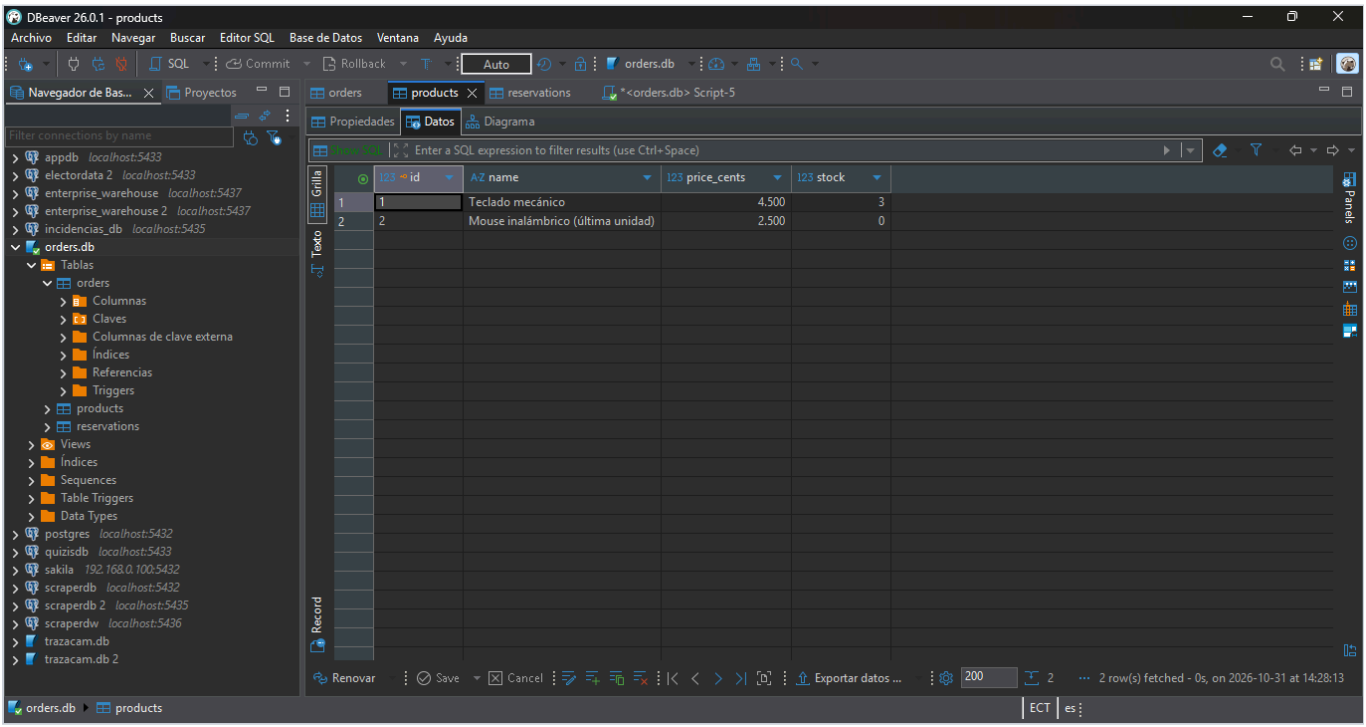


Figura 28. Tabla products: stock final 3 y 0

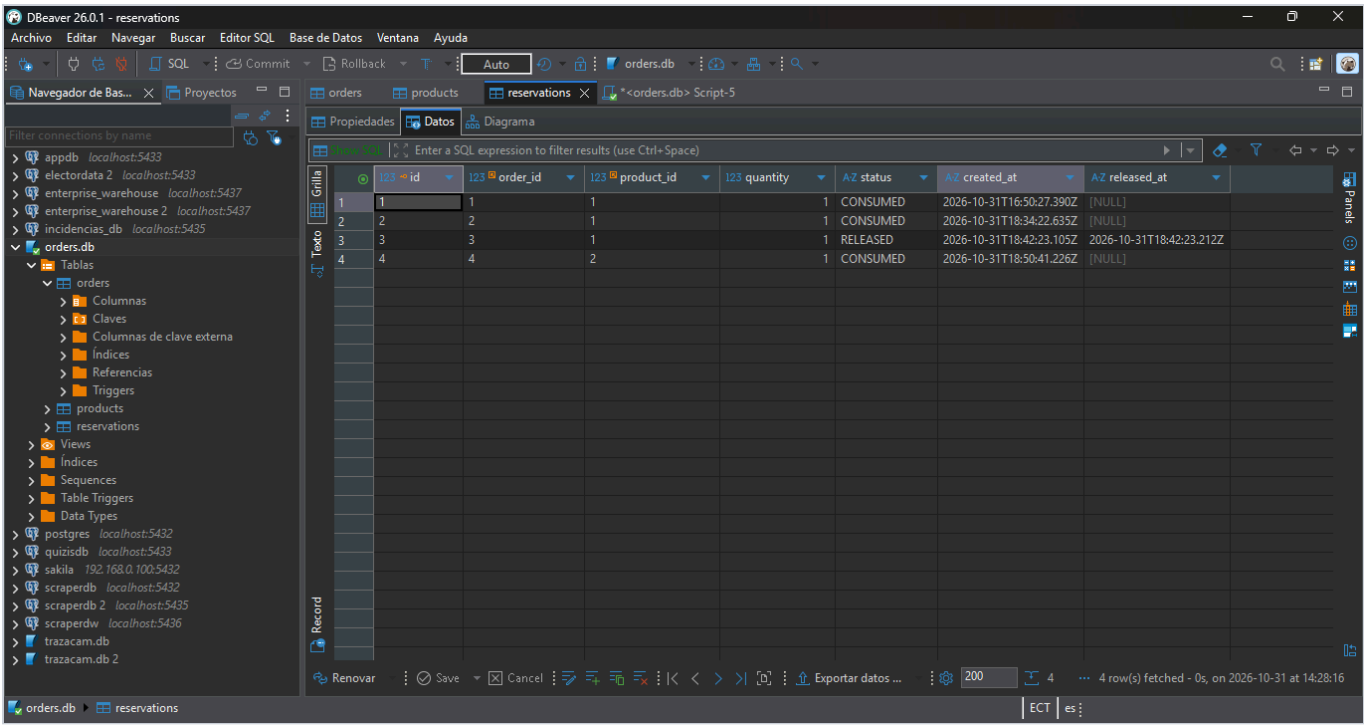


Figura 29. Tabla reservations: una reserva por pedido

4.1 Duplicados: las tres consultas deben devolver 0 filas

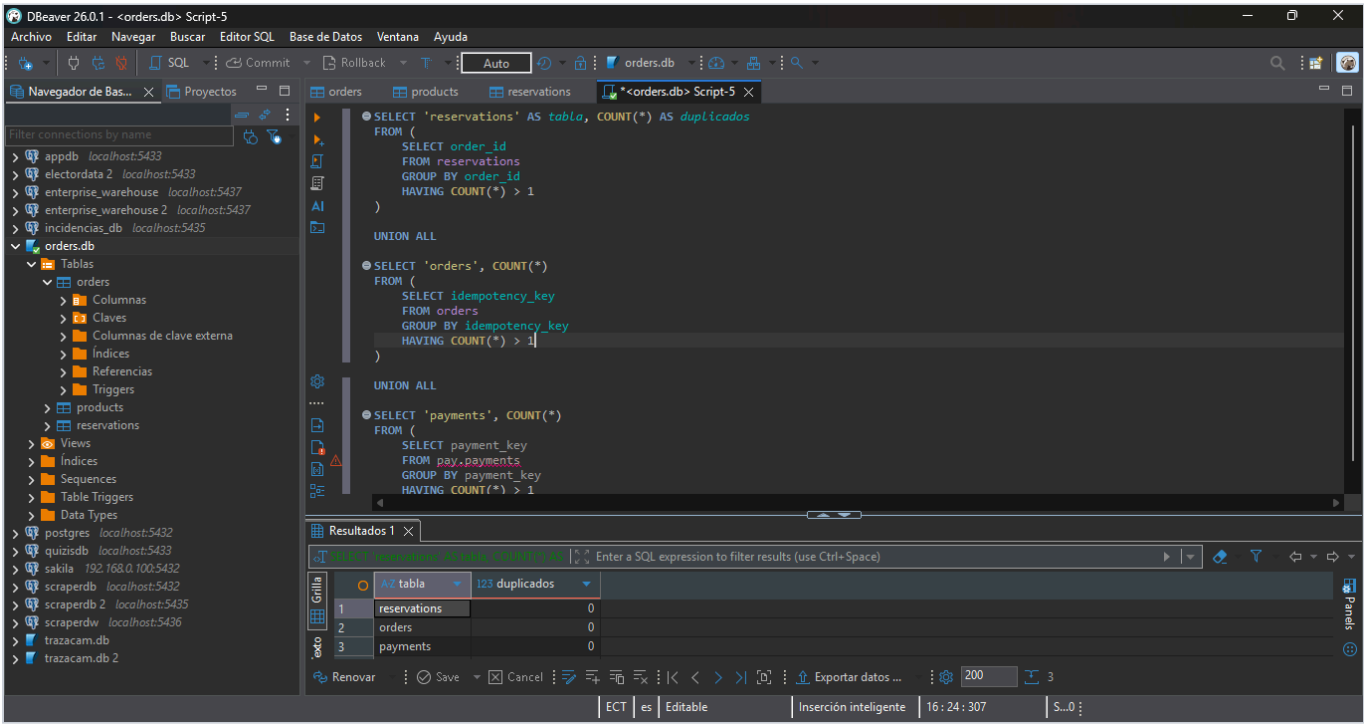


Figura 30. Verificación de duplicados: 0 en reservations, orders y payments

4.2 Efectos por pedido: como máximo 1 reserva y 1 cobro por clave

Aunque se enviaron varios `POST /orders` repetidos con la misma clave (casos 2, 3 y 4), solo existen 5 pedidos: los reenvíos y el intento con otro cuerpo (rechazado con 422) no crearon filas nuevas.

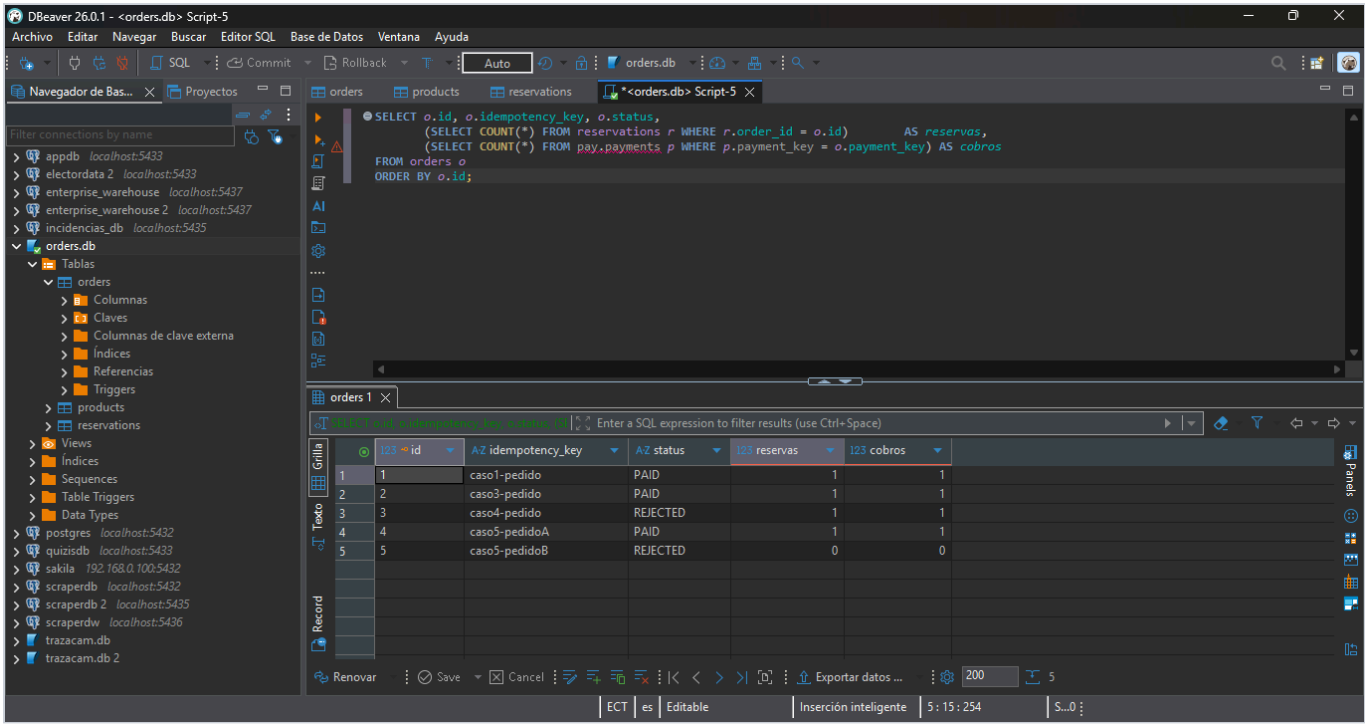


Figura 31. Efectos por pedido: como máximo 1 reserva y 1 cobro

4.3 Pedidos con su reserva (orders.db)

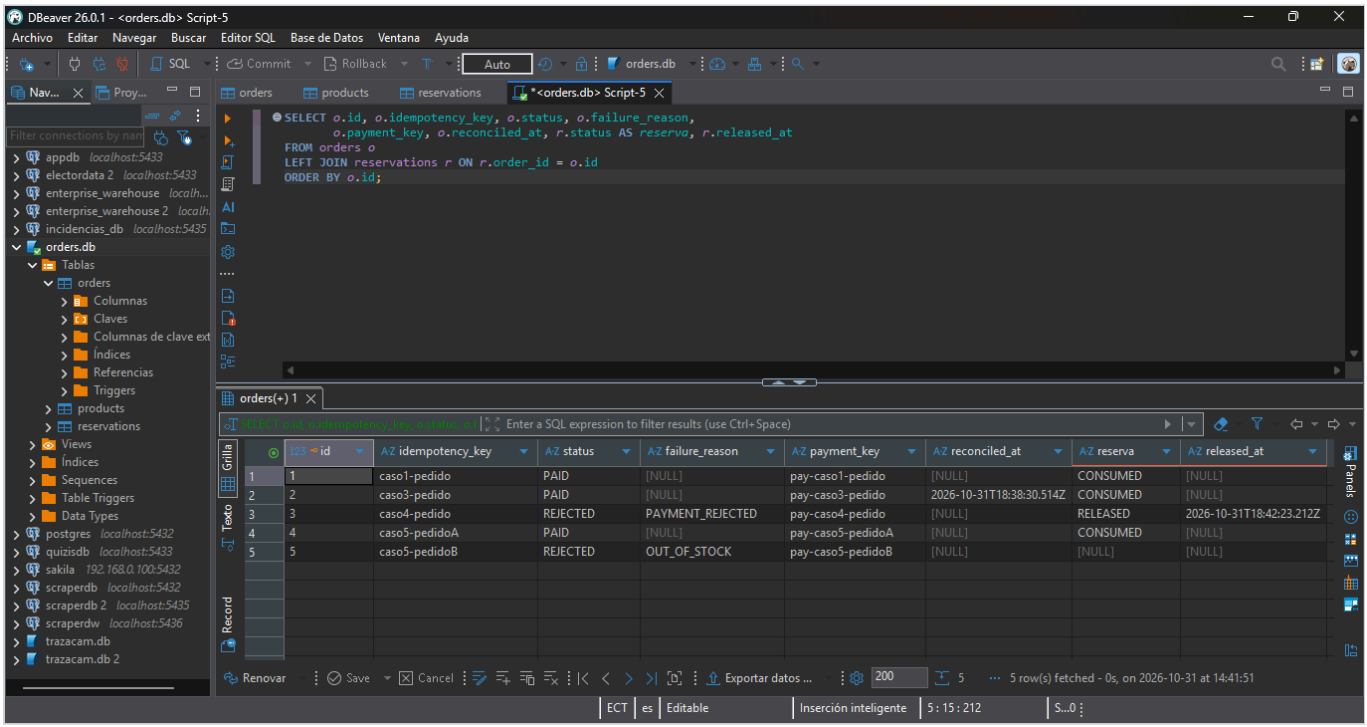


Figura 32. Pedidos con su reserva

4.4 Pagos registrados (payments.db)

El cobro del pedido 2 se registró a las 18:34:22.695, antes del timeout. La Order API no recibió la respuesta, se reinició y a las 18:38:30.514, al reintentar con la misma clave, reconcilió consultando ese mismo registro sin crear otro. El pedido 5 (clave B) no tiene cobro.

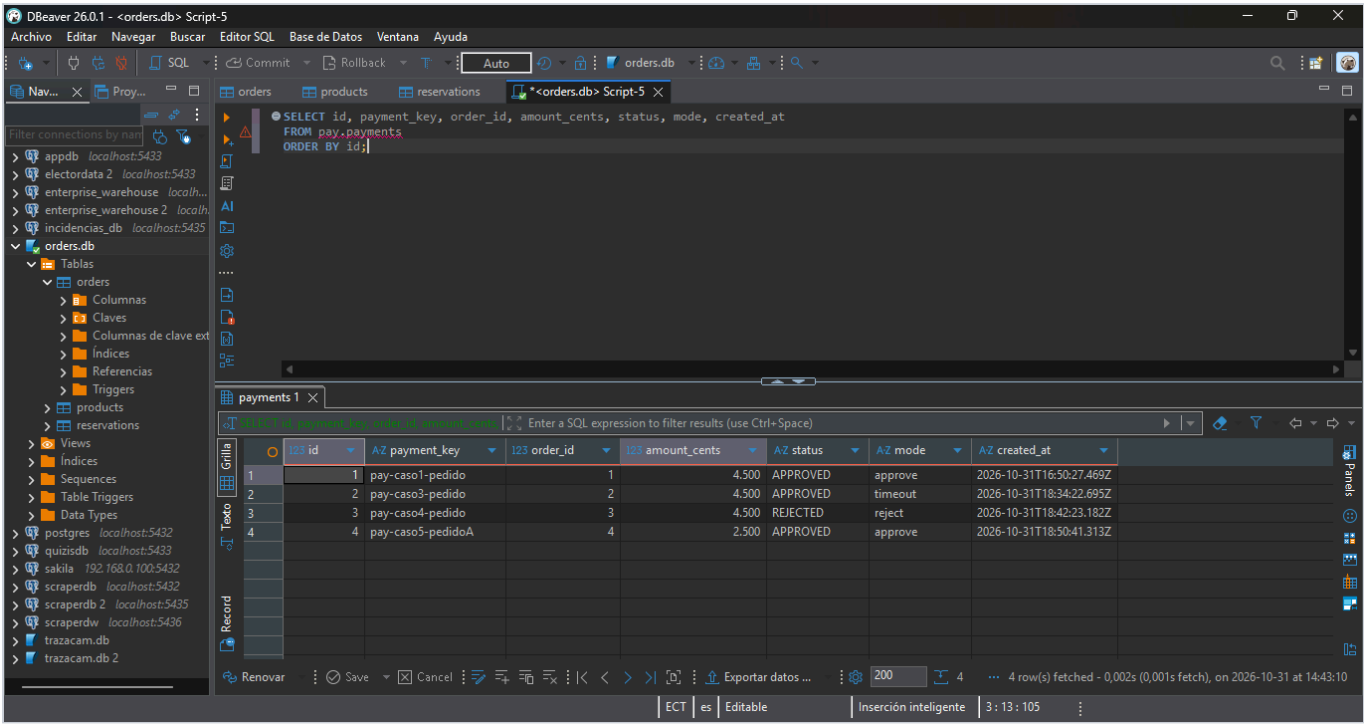


Figura 33. Pagos registrados en el mock: 4 cobros, uno por payment_key

4.5 Stock final

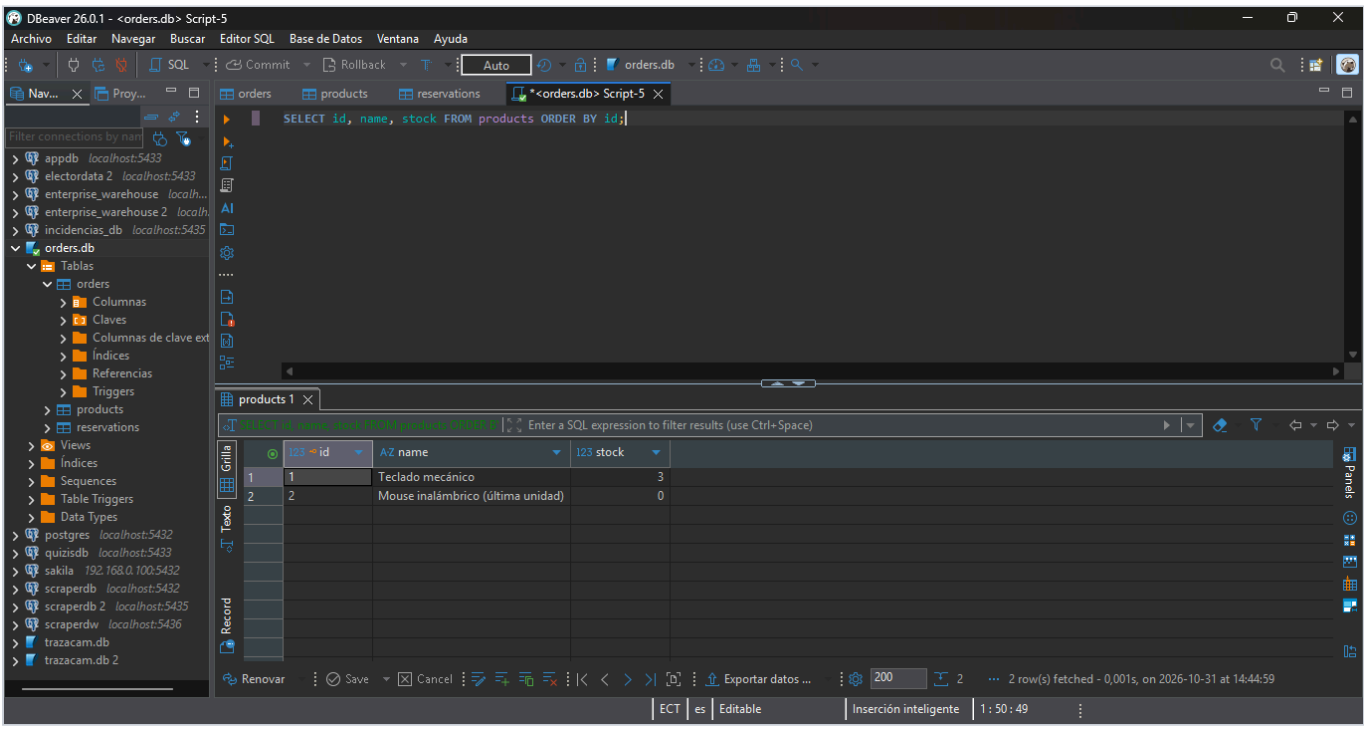


Figura 34. Stock final

5. Conclusión

¿Qué estado persistieron para recuperarse?

En `orders.db` guardamos la `Idempotency-Key` (única), el hash del cuerpo, el **estado del pedido** (incluido el estado intermedio `PAYMENT_UNCERTAIN`), la **`payment_key` estable** derivada de la clave y la reserva con su propio estado (`RESERVED` / `CONSUMED` / `RELEASED`). En `payments.db`, el mock guarda el resultado de cada cobro bajo su `payment_key` única, y hace commit *antes* de responder. Con eso, después de una respuesta perdida o un reinicio, la Order API sabe qué pedidos quedaron abiertos y a qué clave preguntar: consulta `GET /payments/{payment_key}` y aplica el resultado guardado, en lugar de volver a cobrar a ciegas.

¿Cómo comprobaron que no hubo efectos duplicados?

En tres niveles:

1. **Restricciones en la base:** `UNIQUE` en `orders.idempotency_key`, `reservations.order_id` y `payments.payment_key`; `CHECK (stock >= 0)`.
2. **Updates condicionales:** `stock = stock - q WHERE stock >= q` para reservar, y `WHERE status = 'RESERVED'` para que la compensación devuelva stock una sola vez.
3. **Verificación:** 99 assertions de Postman, que comprueban mismo `id`, header de replay, stock y cantidad de cobros después de cada reintento. Además, las consultas `GROUP BY … HAVING COUNT(*) > 1` y la de inconsistencias devuelven 0 filas, y las pruebas de concurrencia producen un solo pedido y un solo cobro.

> No prometemos "exactamente una vez" en la red: un cobro puede enviarse más de una vez. Garantizamos **como máximo un efecto por clave** y que el estado real siempre se puede averiguar después de una falla.